

MBA EM **ENGENHARIA DE SOFTWARE**

Bancos de Dados Não-Relacionais I

Profa. Dra. Kelly Rosa Braghetto

A responsabilidade pela idoneidade, originalidade e licitude dos conteúdos didáticos apresentados é do professor.

Proibida a reprodução, total ou parcial, sem autorização.

[Lei nº 9610/98](#)

Introdução aos Sistemas NoSQL

Baseada no livro: Sadalage, Pramod J., and Martin Fowler. *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Pearson Education

NoSQL – a Origem

- 1998 (primeira aparição)
 - Carlo Strozzi
 - SGBD **Relacional** leve, para SOs derivados do UNIX
 - Não implementava a SQL
- 2009 (reintrodução)
 - Eric Evans (da Rackspace), em evento organizado por Johan Oskarsson (da Last.fm)
 - Popularidade do BigTable/MapReduce (Google), do DynamoDB (Amazon), do Cassandra (Facebook)...
 - Novos sistemas de banco de dados distribuídos e de código aberto

NoSQL – o Nome

- “SQL” no termo atual se refere a SGBD **relacional**
- “NoSQL” $\neq$ não usar a linguagem SQL
- “NoSQL” $\rightarrow$ não usar um SGBD relacional
 - Seria melhor chamar de “NoRel”
- Na última década, reconheceu-se que:
 - Nem todo problema de gerenciamento e análise de dados é melhor solucionado usando um SGBDR tradicional

NoSQL – Diminuindo a Polêmica

- Uma interpretação bastante aceita na atualidade:
 - **Nem todo problema de gerenciamento e análise de dados é melhor solucionado usando exclusivamente um SGBDR tradicional**
- **NoSQL = “Not Only SQL”**

NoSQL – o Nome

- “SQL” no termo atual se refere a SGBD **relacional**
- “NoSQL” $\neq$ não usar a linguagem SQL
- “NoSQL” $\rightarrow$ não usar um SGBD relacional
 - Seria melhor chamar de “NoRel”
- Na última década, reconheceu-se que:
 - Nem todo problema de gerenciamento e análise de dados é melhor solucionado usando um SGBDR tradicional

NoSQL – Diminuindo a Polêmica

- Uma interpretação bastante aceita na atualidade:
 - **Nem todo problema de gerenciamento e análise de dados é melhor solucionado usando exclusivamente um SGBDR tradicional**
- **NoSQL = “Not Only SQL”**

SGBDR ou não SGBDR – Eis a Questão...

- Um SGBDR tradicional provê
 - Eficiência
 - Confiabilidade
 - Conveniência
 - Armazenamento e acesso seguros
 - (para multiusuários)
- para quantidades massivas de dados persistentes

Sobre a Conveniência

- Modelo de dados simples (relacional)
- Linguagem de consulta declarativa (SQL)
- Garantias transacionais
- **Todas essas características parecem ótimas!**
- **Por que então, para algumas aplicações, é vantajoso abrir mão do SGBDR?**

Segundo Michael Stonebraker (*)...

- Há duas razões principais que motivam a troca de um SGBDR por um sistema NoSQL:
 - **Necessidade de um desempenho melhor**
 - **Necessidade de mais flexibilidade**

(*) em “SQL Databases vs. NoSQL Databases”, Communications of the ACM, abril de 2010.

<http://dl.acm.org/citation.cfm?id=1721659>

Detalhe importante: o Stonebraker é o ganhador do prêmio Turing de 2015!

Sobre a Conveniência

- As “conveniências” são garantidas por funcionalidades que vêm embaladas em um pacote indivisível num SGBDR.
- E elas têm os seus respectivos custos.
 - Uma aplicação não consegue “abrir mão” de alguma(s) dessas conveniências, a fim de se livrar do custo associado.

Dados X Modelo Relacional

- Dados ficam organizados em relações
- Há uma álgebra “compreensível” sobre relações
 - Base da linguagem de consulta (a SQL)
 - Tudo se encaixa muito bem em um mesmo pacote!
- Problema: E quando os dados não se encaixam muito bem nesse pacote? Ou seja, o que acontece quando os dados não são relacionais?
 - Antes que possam ser carregados no BD relacional, os dados terão que passar por um processo de reorganização

O Poder (e o Custo) da SQL

- A SQL é uma linguagem de consulta e modificação de dados poderosa. Ela inclui:
 - Seleções, projeções, junções, agregações, operações sobre conjuntos, predicados
- **Às vezes, a SQL oferece muito mais funcionalidades do que uma dada aplicação precisa de fato**
 - Exemplo: há aplicações que só precisam fazer recuperações simples, baseadas na chave do registro
 - Nesses casos, usar um sistema que implementa uma linguagem de consulta complicada é um custo alto demais para se arcar

A Importância (e o Custo) das Garantias Transacionais

- Transações são muito importantes quando há vários usuários acessando os mesmos dados e os requisitos de consistência são rígidos
- Mas em algumas aplicações em que esses requisitos são menos rígidos, até mesmo as garantias mais fracas impostas pelos SGBDRs podem não ser apropriadas

Propriedades ACID de Transações

SGBDRs implementam um modelo transacional que garante as seguintes propriedades:

- **Atomicidade** – uma transação deve ser realizada em sua totalidade ou não ser realizada de forma alguma
- **Consistência** – uma transação deve preservar consistência, levando o BD de um estado consistente para outro, se for completamente executada do início ao fim e sem interferência de outras transações
- **Isolamento** – a execução de uma transação não deve ser interferida por quaisquer outras transações que ocorram simultaneamente
- **Durabilidade** (ou Permanência) – as mudanças aplicadas ao BD por uma transação confirmada precisam persistir no BD (ou seja, não podem ser perdidas em caso de falhas)

Confiabilidade

- Confiabilidade é algo que queremos em qualquer sistema de BD
- Mas é possível lidar com ela de forma diferente em aplicações executadas em modo “batch” ou aplicações de análise de dados
 - Nesses casos, na ocorrência de uma falha, uma estratégia viável para garantir a confiabilidade é simplesmente refazer o processamento todo
 - Essa estratégia não é aplicável, por exemplo, às transações “online” realizadas em um site de vendas de produtos que interage com um SGBD relacional

Dados em Grande Volume

- É uma das razões para o surgimento dos sistemas NoSQL
- **O volume dos dados manipulados hoje em dia é muito maior do que o volume para o qual os SGBDRs tradicionais foram projetados**
- Alguns motivos para o aumento do volume de dados:
 - Grande queda do custo de dispositivos de armazenamento secundário
 - Coletas de dados em altas taxas, feitas por sensores em variados tipos de dispositivos (como celulares, câmeras, etc.) e por web sites (como Facebook, Twitter, etc.)

Eficiência

- **As aplicações da atualidade têm requisitos de desempenho que são muito mais rígidos que antigamente**
- Para aplicações web, eficiência no tempo de resposta é crucial
 - Milhões (e até bilhões) de registros
 - Mesmo para consultas envolvendo operações complexas sobre essa quantidade de registros, o tempo da resposta deve ser menos de 1 segundo

+ Dados, + Flexibilidade, + Eficiência

- Principais motivações para o desenvolvimento dos sistemas NoSQL
- **A proposta dos sistemas NoSQL é “sacrificar” alguns dos benefícios providos pelos SGBDRs tradicionais** em prol de mais:
 - capacidade de armazenamento
 - flexibilidade na representação dos dados
 - eficiência de acesso ao dados

Um parênteses – de volta ao Stonebraker

SQL Databases vs. NoSQL Databases”, Communications of the ACM, abril de 2010

Principais sobrecargas de SGBDs relacionais:

- *Logging*: tudo é gravado duas vezes → uma no disco e outra no *log* do SGBD
- *Locks*: bloqueios para gravação de item de dados (para garantir propriedades transacionais)
- *Latches*: bloqueios especiais, para alteração de estruturas de dados auxiliares do SGBD (ex.: tabela de bloqueios)
- Gerenciamento de *buffer*:
- páginas de disco ↔ memória principal

Sistemas NoSQL – uma Definição Mais Completa

- São uma alternativa aos SGBDs relacionais tradicionais, para lidar com grandes quantidades de dados
- Possuem como **vantagens** sobre o SGBDRs tradicionais:
 - **Esquema mais flexível**
 - **Inicialização mais rápida e barata**
 - **Escalabilidade para grandes volumes de dados (tanto para armazenamento, quanto para a eficiência no acesso aos dados)**
 - **Consistência relaxada, que resulta em melhor desempenho e disponibilidade**

Sistemas NoSQL – uma Definição Mais Completa

- Possuem como **desvantagens** sobre o SGBDRs tradicionais:
 - **A ausência de uma linguagem de consulta declarativa e padronizada**
 - Implica em maior esforço para os desenvolvedores, que precisam implementar as operações de manipulação dos dados usando linguagens de programação
 - **Consistência relaxada**
 - Menos garantias com relação à consistência dos dados

Sistemas NoSQL – Resumo das Características

- + Esquema mais flexível
- + Inicialização mais rápida e barata
- + Escalabilidade para grandes volumes de dados
- + Consistência relaxada → melhor desempenho e alta disponibilidade
- Ausência de linguagens de consulta declarativas → mais programação
- Consistência relaxada → menos garantias

Seis Características Comuns de Sistemas NoSQL (*)

- 1) A habilidade de **escalar horizontalmente** *operações simples* em vários servidores
- 2) A habilidade de **replicar e distribuir (particionar) dados** em vários servidores
- 3) Substituição da “**comunicação**” via SQL por **APIs simples**
- 4) Um modelo de **controle de concorrência mais fraco** que o das transações ACID usadas nos SGBDRs
- 5) Uso eficiente de **índices distribuídos e memória RAM** para armazenamento dos dados
- 6) A habilidade de adicionar dinamicamente novos atributos aos registros de dados (**ausência de esquema fixo** – *schemaless*)

(*) Segundo Rick Cattell em “Scalable SQL and NoSQL Data Stores”, SIGMOD Record, December 2010

Definições Importantes

- “Operações simples”
 - Busca de um registro por meio de sua chave (*key lookup*), leituras e escritas de um registro ou de um número pequeno de registros
 - Constituem o tipo de operação mais frequente nas aplicações web atuais
- “Escalabilidade horizontal”
 - Habilidade de distribuir os dados e o processamento de operações simples em vários servidores (nós), sem o compartilhamento de RAM ou disco entre os servidores → **arquitetura “*shared nothing*”**
 - Esse tipo de escalabilidade geralmente é mais barata que a vertical, principalmente quando usa *commodity hardware*

Obs.: Escalabilidade vertical → ocorre quando o SGBD usa vários *cores* que compartilham a mesma RAM e disco, ou seja, o SGBD está em uma única máquina

Escalabilidade Horizontal

- Pode ser feita de duas maneiras:
 - **Particionando os dados (*sharding*)** entre os nós
 - **Particionamento horizontal:** cada nó armazena um subconjunto das “linhas” do BD
 - **Particionamento vertical:** cada nó armazena um subconjunto das “colunas” do BD
 - **Replicando os dados** nos nós
- Possibilita que um grande número de operações simples (principalmente consultas) sejam realizadas a cada segundo → **paralelização**

Replicação de Dados

- Pode ter dois objetivos diferentes (e não mutuamente exclusivos):
 - Tolerância a falhas
 - Balanceamento de carga de operações de acesso a dados
- A propagação das atualizações entre as réplicas pode ser:
 - **Assíncrona** → garante “**eventual consistency**”: não há a garantia de que um dado lido é o mais atual, mas as atualizações serão propagadas para todos os nós em algum momento do tempo
 - **Síncrona** → garante consistência
- Quando a replicação é assíncrona, uma falha em um nó pode causar perda irreversível de dados!

Armazenamento dos Dados

- Os dados e os índices podem ser mantidos
 - No disco (HD, SSD)
 - Na memória RAM, com tolerância a falhas por meio de persistência em disco ou replicação

Problemas de Consistência

- Tipos de conflitos que podem ocorrer em SGBDRs:
 - **Escrita-escrita** → quando dois clientes tentam escrever sobre os mesmos dados ao mesmo tempo
 - **Leitura-escrita** → quando um cliente lê dados inconsistentes no meio da escrita de outro cliente
- Sistemas distribuídos também têm esses conflitos. Ex.:
 - Alterações simultâneas em cópias diferentes de um dado registro
 - Leitura de uma cópia desatualizada

Conflitos e Consistência

- Há duas abordagens para evitar conflitos:
 - **Pessimista** → bloqueia os registros de dados, para evitar os conflitos
 - **Otimista** → detectam os conflitos e depois os tratam
- Para obter boa consistência, é preciso envolver muitos nós nas operações sobre os dados
 - Mas isso aumenta a latência (tempo de resposta das operações)

Teorema CAP

[Brewer, 2000; Lynch & Gilbert, 2002]

Consistency X availability X partition tolerance

Segundo o teorema, em um sistema distribuído pode-se garantir apenas **duas dessas propriedades por vez**:

- **Consistência** dos valores nas cópias de itens replicados
- **Disponibilidade (*Availability*)** – tem um significado diferente do usual: "toda requisição recebida por um nó operante no sistema precisa resultar em uma resposta"
 - Portanto, um nó falho (inoperante) não indica falta de disponibilidade CAP
- **Tolerância à partição** – o sistema continuar operante em caso de uma falha na rede que impossibilite a comunicação entre dois ou mais conjuntos de nós

Teorema CAP

[Brewer, 2000; Lynch & Gilbert, 2002]

Consistency X availability X partition tolerance

- Para ter um *cluster* C-A, é preciso garantir que o sistema todo fique inoperante em uma falha de partição de rede
 - Como isso é muito caro para *data centers*, não é desejável abrir mão da tolerância à partição de rede
- Portanto, em sistemas de dados distribuídos, é preciso **relaxar** a Consistência em prol de maior Disponibilidade (ou vice-versa)
 - Porém, não é preciso abrir mão completamente de uma delas

Modelo de Consistência BASE

Sistemas NoSQL que seguem o modelo BASE toleram inconsistências temporárias de dados a fim de priorizar a disponibilidade e desempenho:

- **Basically Available:** o sistema sempre responde a requisições, mesmo que a resposta não reflita o estado mais recente dos dados
- **Soft state:** o estado do sistema pode mudar ao longo do tempo, mesmo sem novas entradas externas
 - Razões: replicação assíncrona, sincronização entre nós, recuperação
- **Eventually consistent:** garante que a consistência será obtida assim que todas as escritas forem propagadas para todos os nós
 - Todos os nós acabam vendo o mesmo valor, mas não necessariamente agora

SGBD Relacionais Também Podem ser Escaláveis

- É possível fazer particionamento e replicação de dados em SGBDRs
- Os SGBDRs, desde sua criação, já foram especializados para diferentes necessidades:
 - Data Warehouses (operações de leitura predominam)
 - BDs “in-memory”
 - BDs distribuídos
 - **BDs escaláveis horizontalmente**
 - Veja: MySQL Cluster, VoltDB, NuoDB, Clustrix
- Se o desempenho de um SGBDR for “competitivo” com o de um sistema NoSQL, porque abriríamos mão dos benefícios de se ter SQL + consistência ACID?

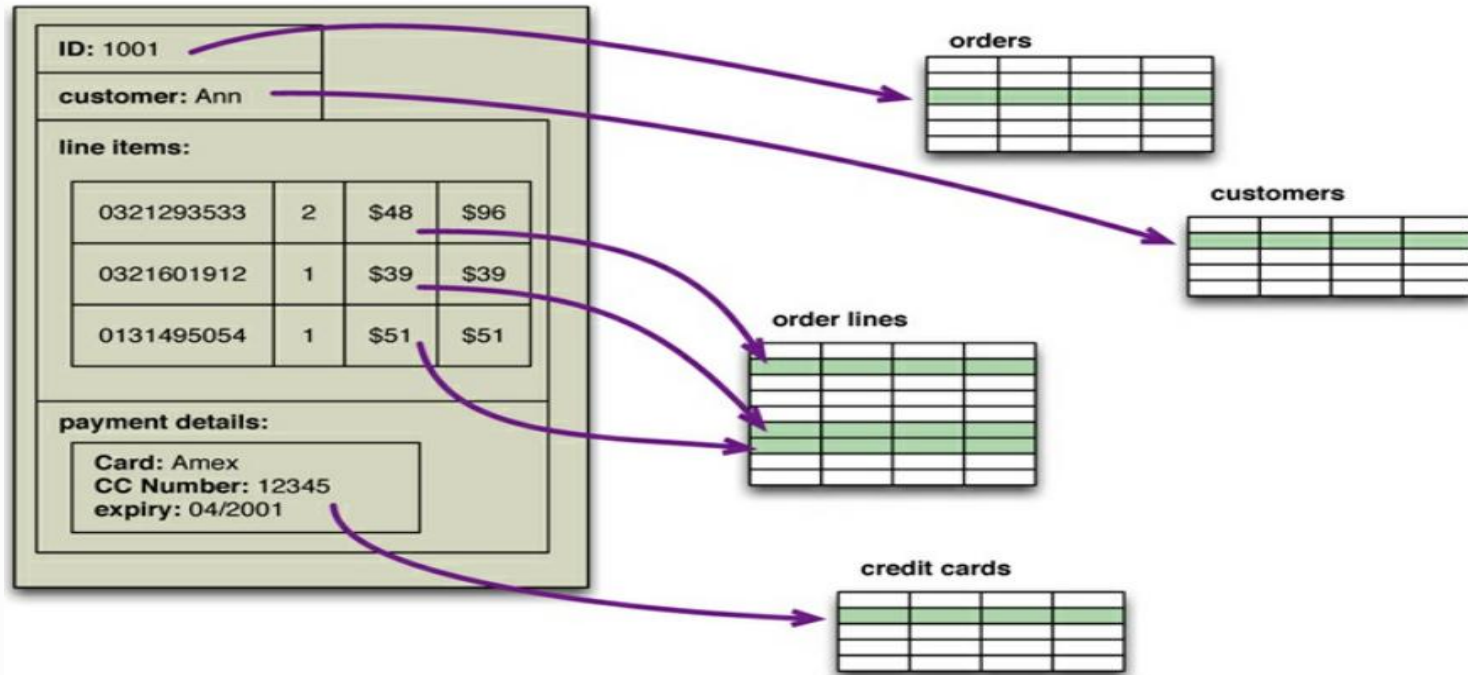
Modelos de Dados NoSQL – Modelos Baseados em Agregados

Baseado no livro *NoSQL Distilled – A Brief Guide to the Emerging World of Polyglot Persistence*, de Pramod Sadalage e Martin Fowler

Contexto: Bancos de Dados Relacionais

- Modelo relacional: 1970
 - Primeiras implementações: a partir 1974
- Um **BD relacional** é um conjunto de **relações**
- Cada **relação** é um conjunto de **tuplas**
- Cada **tupla** é um conjunto de valores para **atributos**
 - Atributos são sempre simples e atômicos
- Uma tupla pode representar uma **entidade** ou **relacionamento**
- Um relacionamento é expresso por um atributo que **referencia** uma tupla da mesma relação ou de uma outra relação
- A SQL é a linguagem de consulta padrão para BDs relacionais

Modelo de Dados Relacional – Limitações



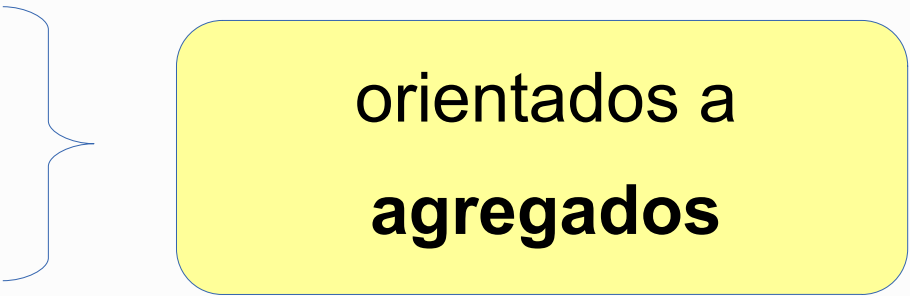
- **Impedance Mismatch:** Diferença entre como os dados são modelados na aplicação e como eles são armazenados no banco de dados relacional
- Não projetado para uso em aglomerados de máquinas (*clusters*)

• Anos 2000: demanda crescente por mais flexibilidade e escalabilidade, principalmente para dados dos sistemas web

Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Sistemas NoSQL – Modelos de Dados

- Diferentes tipos de modelos de dados são usados nos sistemas NoSQL
 - Grande heterogeneidade
- Esses modelos costumam ser divididos nas seguintes categorias:
 - **Chave-valor**
 - **Documentos**
 - **Famílias de colunas**
 - **Grafos**



orientados a
agregados

Agregados

Agregado: conjunto de objetos relacionados, tratados como uma **unidade de dados**

- Origem: *Domain Driven Design*
 - Abordagem para desenvolvimento de software
- Um agregado pode ter uma estrutura mais complexa que a de uma tupla relacional, pois permite **aninhamento**
- Os agregados são as **unidades para manipulação** dos dados e **gerenciamento da consistência** nos sistemas NoSQL
 - A comunicação com o componente de armazenamento dos dados é feita em termos de agregados
 - A modificação de um agregado é implementada como uma operação atômica

Agregados

Agregados facilitam o funcionamento dos Sistemas NoSQL em aglomerados de máquinas (*clusters*)

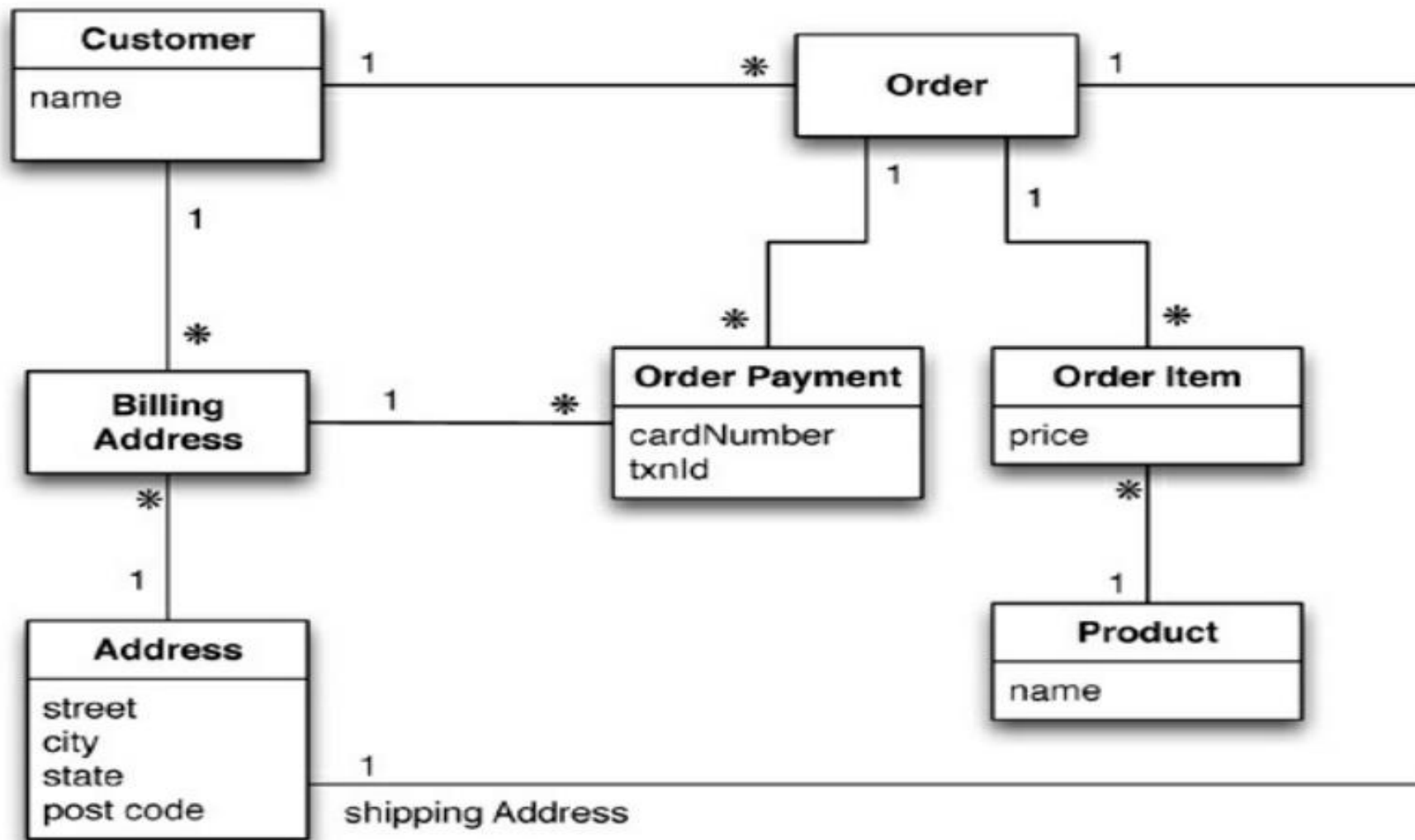
- Nesses sistemas, deseja-se minimizar o número de nós que precisam ser acessados para se responder uma consulta
- Um agregado indica quais dados são manipulados juntos e que, portanto, deveriam ficar em um mesmo nó
 - Unidade natural para replicação e particionamento
- Agregados também facilitam o desenvolvimento de aplicações
 - Possuem uma estrutura mais próxima da dos objetos de dados em memória manipulados por elas

Exemplo: Comércio Eletrônico

Necessidade de armazenar dados de:

- Clientes
- Catálogo de produtos
- Pedidos
- Endereços de entrega
- Endereços de faturamento
- Pagamentos

Exemplo: Esquema Relacional do BD (em UML)



Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Exemplo: o BD Relacional

Customer	
Id	Name
1	Martin

Orders		
Id	CustomerId	ShippingAddressId
99	1	77

Product	
Id	Name
27	NoSQL Distilled

BillingAddress		
Id	CustomerId	AddressId
55	1	77

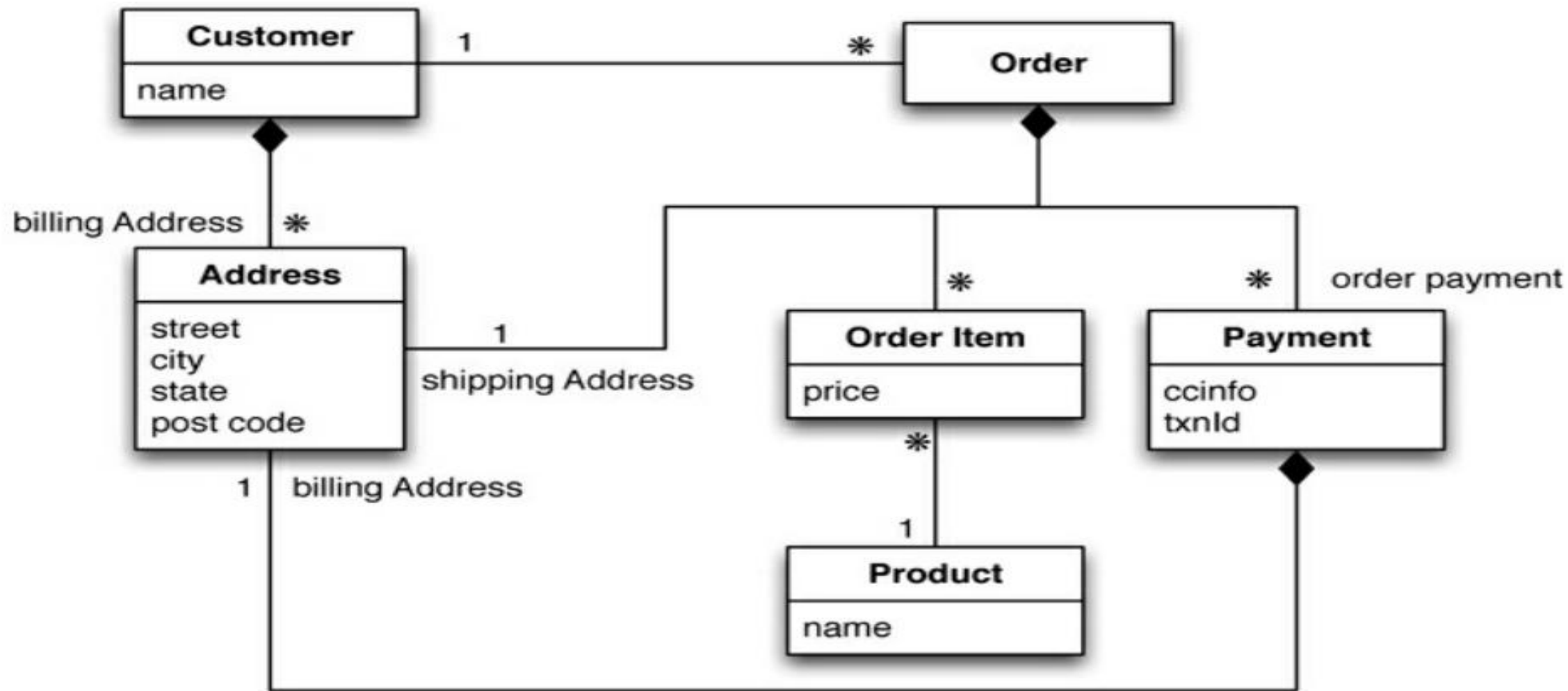
OrderItem			
Id	OrderId	ProductId	Price
100	99	27	32.45

Address	
Id	City
77	Chicago

OrderPayment				
Id	OrderId	CardNumber	BillingAddressId	txnId
33	99	1000-1000	55	abelif879rft

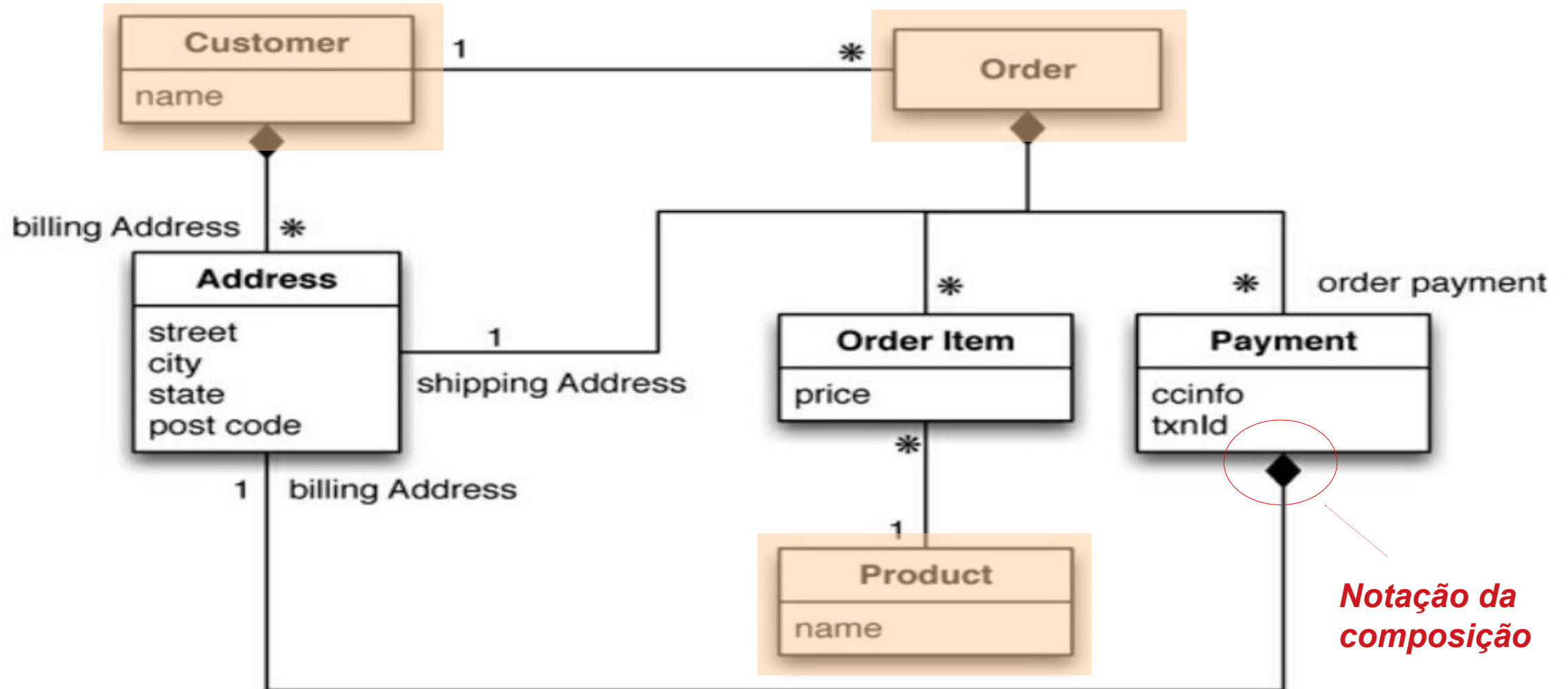
Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Exemplo: Esquema de Agregados (em UML)



Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Exemplo: Esquema de Agregados (em UML)



- **Três tipos de agregados: *Customer*, *Order* e *Product***

Instâncias do Esquema de Agregados

```
// in customers
{
  "id":1,
  "name":"Martin",
  "billingAddress":[{"city":"Chicago"}]
}

// in orders
{
  "id":99,
  "customerId":1,
  "orderItems":[
    {
      "productId":27,
      "price": 32.45,
      "productName": "NoSQL Distilled"
    }
  ],
  "shippingAddress":[{"city":"Chicago"}]
  "orderPayment":[
    {
      "ccinfo":"1000-1000-1000-1000",
      "txnId":"abelif879rft",
      "billingAddress": {"city": "Chicago"}
    }
  ],
}
```

Denormalização de *Product*

Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Ainda sobre o Exemplo

**Como ocorre em qualquer modelagem de BD,
uma dada modelagem de agregados pode favorecer
algumas operações enquanto atrapalha outras**

- Por exemplo, o agregado “order”:
 - Ajuda quando um usuário precisa consultar os detalhes de um pedido seu ou quando o comerciante está "processando" o pedido para a entrega
 - Recuperando um só agregado se responde à consulta
- Atrapalha quando deseja-se saber o histórico de vendas dos produtos:
 - Para responder essa consulta, pode ser preciso percorrer todos os agregados de pedidos armazenados no BD

Instância do Esquema de Agregados

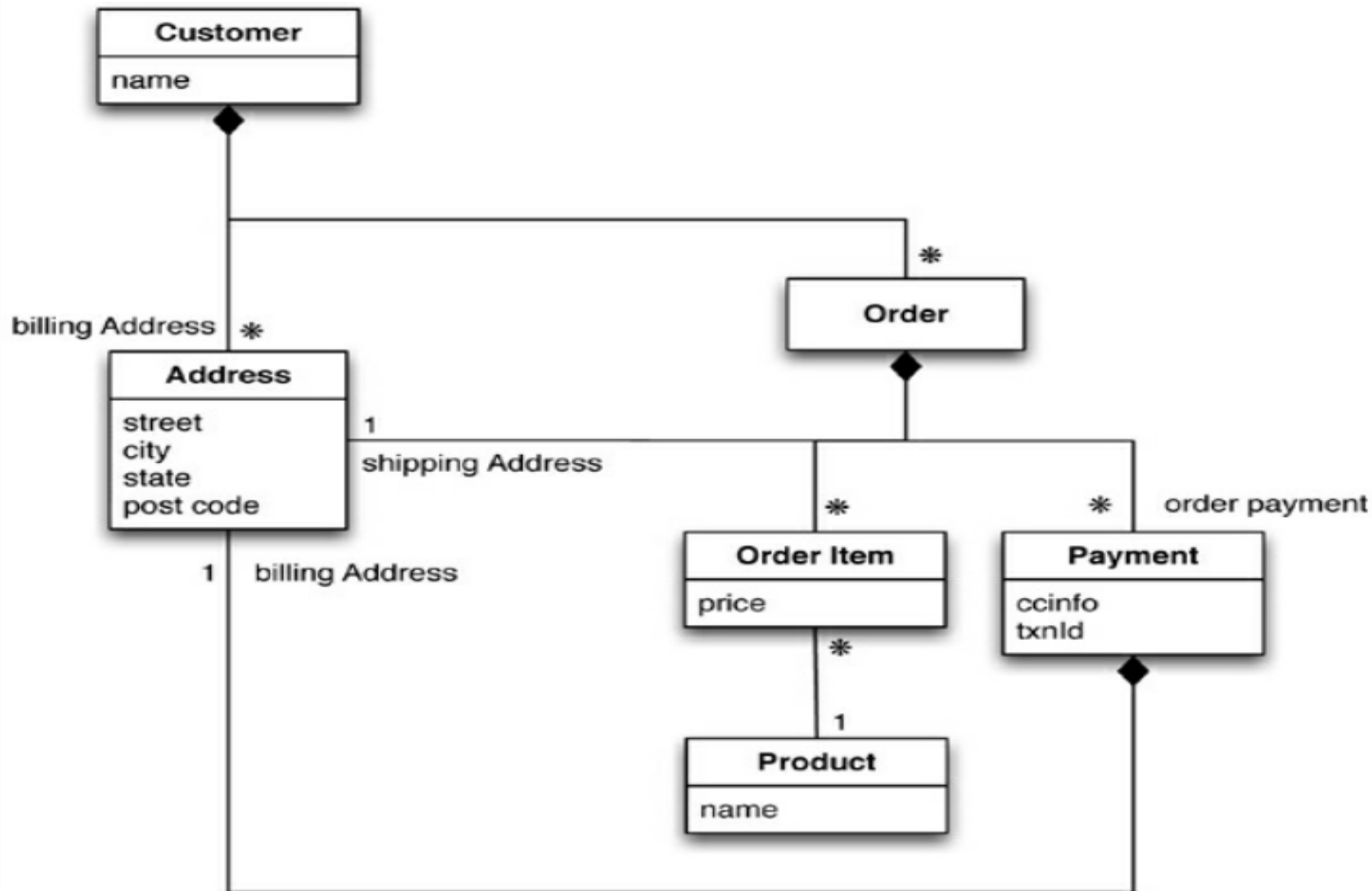
```
// in customers
{
  "id":1,
  "name":"Martin",
  "billingAddress":[{"city":"Chicago"}]
}

// in orders
{
  "id":99,
  "customerId":1,
  "orderItems":[
    {
      "productId":27,
      "price": 32.45,
      "productName": "NoSQL Distilled"
    }
  ],
  "shippingAddress":[{"city":"Chicago"}]
  "orderPayment":[
    {
      "ccinfo":"1000-1000-1000-1000",
      "txnId":"abelif879rft",
      "billingAddress": {"city": "Chicago"}
    }
  ],
}
```

Detalhes dos itens inclusos são recuperados junto com os demais dados do pedido

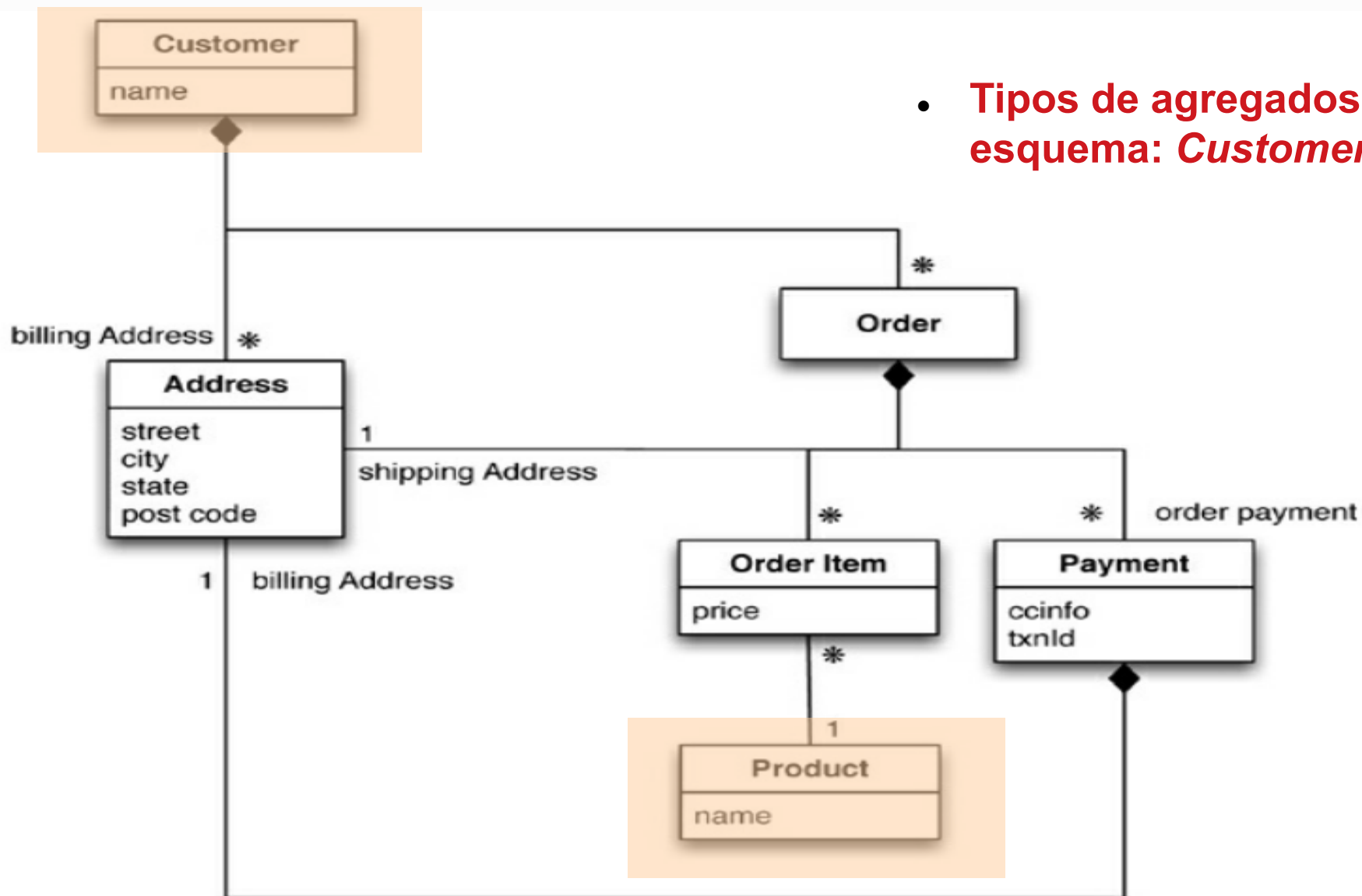
Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Exemplo: Esquema de Agregados Alternativo



Fonte da imagem: NoSQL
Distilled. Sadalage, P. J., &
Fowler, M. (2013)

Exemplo: Esquema de Agregados Alternativo



Fonte da imagem: NoSQL
Distilled. Sadalage, P. J., &
Fowler, M. (2013)

Instância do Esquema de Agregados Alternativo

```
{
  "customer": {
    "id": 1,
    "name": "Martin",
    "billingAddress": [{"city": "Chicago"}],
    "orders": [
      {
        "id": 99,
        "orderItems": [
          {
            "productId": 27,
            "price": 32.45,
            "productName": "NoSQL Distilled"
          }
        ],
        "shippingAddress": [{"city": "Chicago"}],
        "orderPayment": [
          {
            "ccinfo": "1000-1000-1000-1000",
            "txnId": "abelif879rft",
            "billingAddress": {"city": "Chicago"}
          }
        ]
      }
    ]
  }
}
```

→ **Denormalização
de *Products***

Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Relacionamentos e Agregados

- Em BDs relacionais, não é possível distinguir os relacionamentos de agregação dos demais
 - Consequência: SGBDs relacionais não tem conhecimento sobre os agregados para apoiar o armazenamento e distribuição dos dados
- Em BDs orientados a agregados, o foco é a unidade de interação com o gerenciador de armazenamento de dados
 - O agregado não representa uma organização lógica “intrínseca” aos dados
 - Expressa a forma como os dados são usados pelas aplicações

Mais sobre os Agregados

- Não há um conceito de “chave estrangeira” para impor integridade aos relacionamentos entre agregados
- A estrutura interna dos agregados não é “vista” nas decisões sobre armazenamento e distribuição dos dados
- Em muitos sistemas NoSQL, não é possível manipular múltiplos agregados de forma atômica
- Quando há muitas aplicações acessando o mesmo BD NoSQL, pode ser difícil estruturar agregados que beneficiem todas ao mesmo

Agregados – Resumo

- Sistemas NoSQL usam diferentes tipos de modelos de dados
- O modelo empregado pela maioria pode ser generalizado pelo modelo de agregados
- Um agregado reúne objetos relacionados, que precisam ser tratados como uma unidade
- A distribuição, o armazenamento e o controle de consistência em sistemas NoSQL são feitos em termos de agregados
- O modelo de agregados facilita o uso de aglomerados de máquinas

Bancos de Dados Chave-Valor e de Documentos

- São fortemente orientados a agregados
- São constituídos por conjuntos de agregados
 - cada agregado tem uma chave (ou ID), usada na recuperação dos dados
- Diferença entre eles:
 - **BD chave-valor:** os agregados são “opacos” para o SGBD
 - **BD de documentos:** o SGBD pode “enxergar” a estrutura dos agregados

Bancos de Dados Chave-Valor

- A opacidade traz flexibilidade no armazenamento
 - É possível armazenar qualquer tipo de dado dentro de um agregado
 - O SGBD vê o conteúdo de um agregado apenas como um punhado de bits que precisam ser mantidos juntos
 - Mas pode haver limite de tamanho
- Um agregado é acessado por meio de busca pela sua chave
- O suporte à definição de índices sobre campos do agregado geralmente não é implementado

Bancos de Dados Chave-Valor

- **Interface simples**

- Modelo de dados:

pares (chave, valor)

- Operações:

- **Insert**(chave,valor)
 - **Fetch**(chave) → busca/recuperação de um valor
 - **Update**(chave,valor)
 - **Delete**(chave)

- Alguns sistemas permitem que a operação de Fetch seja realizada sobre faixa de valores de chaves
 - Ex.: $2 < \text{chave} < 10$

Sistemas de Bancos de Dados Chave-Valor

Exemplos:

- Redis
- <http://redis.io/>
- Memcached
- <http://memcached.org/>
- Riak
- <http://basho.com/products/riak-kv/>
- OrientDB
- <http://orientdb.com/>

Sistemas de Bancos de Dados Chave-Valor Populares

☐ include secondary database models

71 systems in ranking, February 2026

Rank			DBMS	Database Model	Score		
Feb 2026	Jan 2026	Feb 2025			Feb 2026	Jan 2026	Feb 2025
1.	1.	1.	Redis	Key-value, Multi-model ⓘ	147.04	+2.88	-10.87
2.	2.	2.	Amazon DynamoDB	Multi-model ⓘ	67.87	-1.92	-7.71
3.	3.	3.	Microsoft Azure Cosmos DB	Multi-model ⓘ	23.88	-0.92	+1.75
4.	4.	4.	Memcached	Key-value	14.60	-0.19	-2.09
5.	5.	5.	etcd	Key-value	7.43	-0.19	+0.37
6.	6.	6.	Hazelcast	Key-value, Multi-model ⓘ	5.34	+0.02	-0.17
7.	7.	7.	Aerospike	Multi-model ⓘ	3.66	-0.11	-1.43
8.	↑9.	↑12.	ArangoDB	Multi-model ⓘ	3.47	+0.15	+0.56
9.	↓8.	↑11.	RocksDB	Key-value	3.39	-0.18	+0.33
10.	10.	↓9.	Riak KV	Key-value	3.21	-0.02	-0.13
11.	11.	↓8.	Ehcache	Key-value	3.04	+0.08	-1.28
12.	12.	↓10.	Oracle NoSQL	Multi-model ⓘ	2.90	-0.03	-0.39

<https://db-engines.com/en/ranking/key-value+store>

Bancos de Dados de Documentos

- Vantagem da não-opacidade: flexibilidade no acesso
 - É possível fazer buscas sobre o conteúdo do agregado
 - Também é possível recuperar partes de um agregado (ao invés de recuperá-lo por inteiro)
- O SGBD define quais são as estruturas e tipos de dados permitidos dentro do agregado
 - Há restrições quanto ao que pode ser armazenado no agregado
- É possível definir índices sobre o conteúdo dos agregados

Bancos de Dados de Documentos

- **Interface**

- Modelo de dados: **pares (chave, documento)**
- Documento: dados em formatos semiestruturados (**JSON**, XML, etc.)
 - Permite a definição de documentos aninhados
- Documentos são agrupados em **coleções**

Bancos de Dados de Documentos

- Operações básicas:
- **Insert**(chave,documento), **Fetch**(chave),
- **Update**(chave,documento), **Delete**(chave)
- Também fazem **Fetch de documentos buscando pelo seus conteúdos**
 - mas não há uma linguagem de busca padrão
 - cada SGBD tem sua própria API
- Geralmente não implementam bloqueios explícitos
 - Atomicidade em operações envolvendo mais de um agregado precisa ser implementada nas aplicações

Sistemas de Bancos de Dados de Documentos

Exemplos:

- MongoDB
- <http://www.mongodb.org/>
- Couchbase
- <http://www.couchbase.com/>
- Amazon DynamoDB
- <https://aws.amazon.com/pt/dynamodb/>
- CouchDB
- <http://couchdb.apache.org/>

Sistemas de Bancos de Dados de Documentos Populares

☐ include secondary database models

58 systems in ranking, February 2026

Rank			DBMS	Database Model	Score		
Feb 2026	Jan 2026	Feb 2025			Feb 2026	Jan 2026	Feb 2025
1.	1.	1.	MongoDB	Document, Multi-model 	378.73	+1.99	-17.90
2.	2.	2.	Databricks	Multi-model 	144.51	+2.97	+54.48
3.	3.	3.	Amazon DynamoDB	Multi-model 	67.87	-1.92	-7.71
4.	4.	4.	Microsoft Azure Cosmos DB	Multi-model 	23.88	-0.92	+1.75
5.	5.	 6.	Firebase Realtime Database	Document	14.29	-0.36	+0.97
6.	6.	 5.	Couchbase	Multi-model 	10.79	-0.84	-4.84
7.	7.	 8.	Google Cloud Firestore	Document	9.35	-0.08	+2.44
8.	 9.	 7.	CouchDB	Document, Multi-model 	6.66	+0.58	-1.15
9.	 8.	9.	Realm	Document	6.62	-0.22	-0.12
10.	10.	 11.	MarkLogic	Multi-model 	4.09	-0.17	-0.07
11.	11.	 10.	Aerospike	Multi-model 	3.66	-0.11	-1.43
12.	 13.	 15.	ArangoDB	Multi-model 	3.47	+0.15	+0.56

<https://db-engines.com/en/ranking/document+store>

Chave-Valor ou Documentos?

- Na prática, a separação entre esses dois modelos não é clara
- Exemplos de chave-valor com recursos de documentos:
 - O Riak permite adicionar metadados nos agregados, para apoiar a criação de índices e de relacionamentos entre agregados
 - O Redis permite dividir um agregado em listas ou conjuntos
 - É possível prover consultas baseadas em conteúdo em BDs chave-valor integrando-os a ferramentas de busca, como a Solr e a ElasticSearch
 - <https://lucene.apache.org/solr/>
 - <https://www.elastic.co/pt/elasticsearch/>

Chave-Valor ou Documentos?

A escolha entre um modelo e outro deve ser guiada pelo tipo de consulta que se espera receber com mais frequência no BD

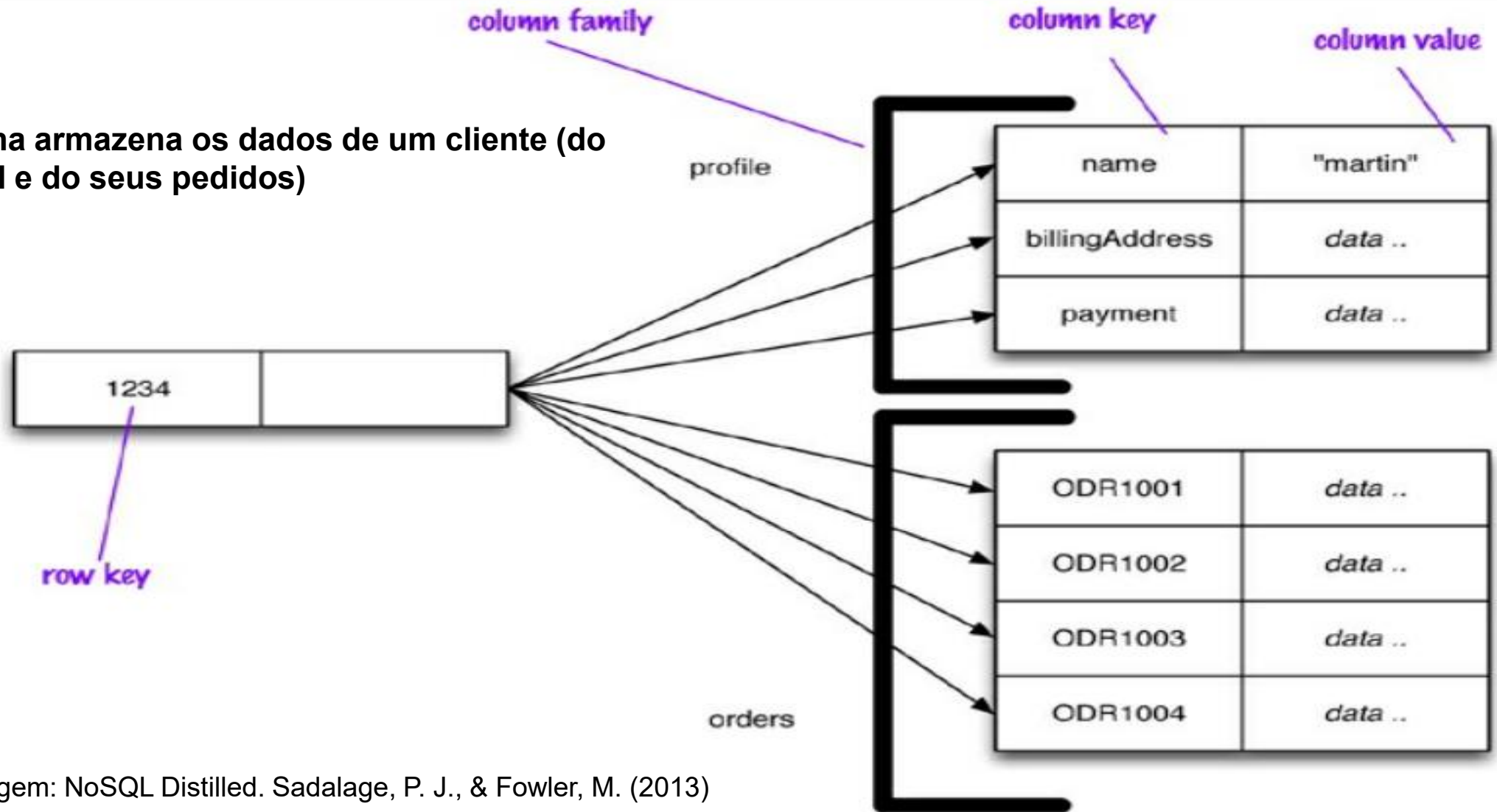
- Em BDs chave-valor:
 - Recuperação de agregados pela chave
- Em BDs de documentos:
 - Recuperação de agregados a partir de consultas baseadas na estrutura interna do documento (e também na chave)

Bancos de Dados de Famílias de Colunas

- **BDs de famílias de colunas** armazenam dados como linhas, onde cada linha possui uma chave e colunas agrupadas em famílias
 - Cada coluna deve fazer parte de uma família
- Dentro de uma mesma família ficam dados que, frequentemente, serão acessados juntos
- As linhas não precisam ter as mesmas colunas
- Novas colunas podem ser adicionadas a qualquer linha em qualquer momento (sem a necessidade de serem adicionadas às demais linhas)
- Uma coluna pode ser uma **supercoluna** (= **mapa de colunas**), ou seja, um contêiner para outras colunas

Famílias de Colunas – Exemplo

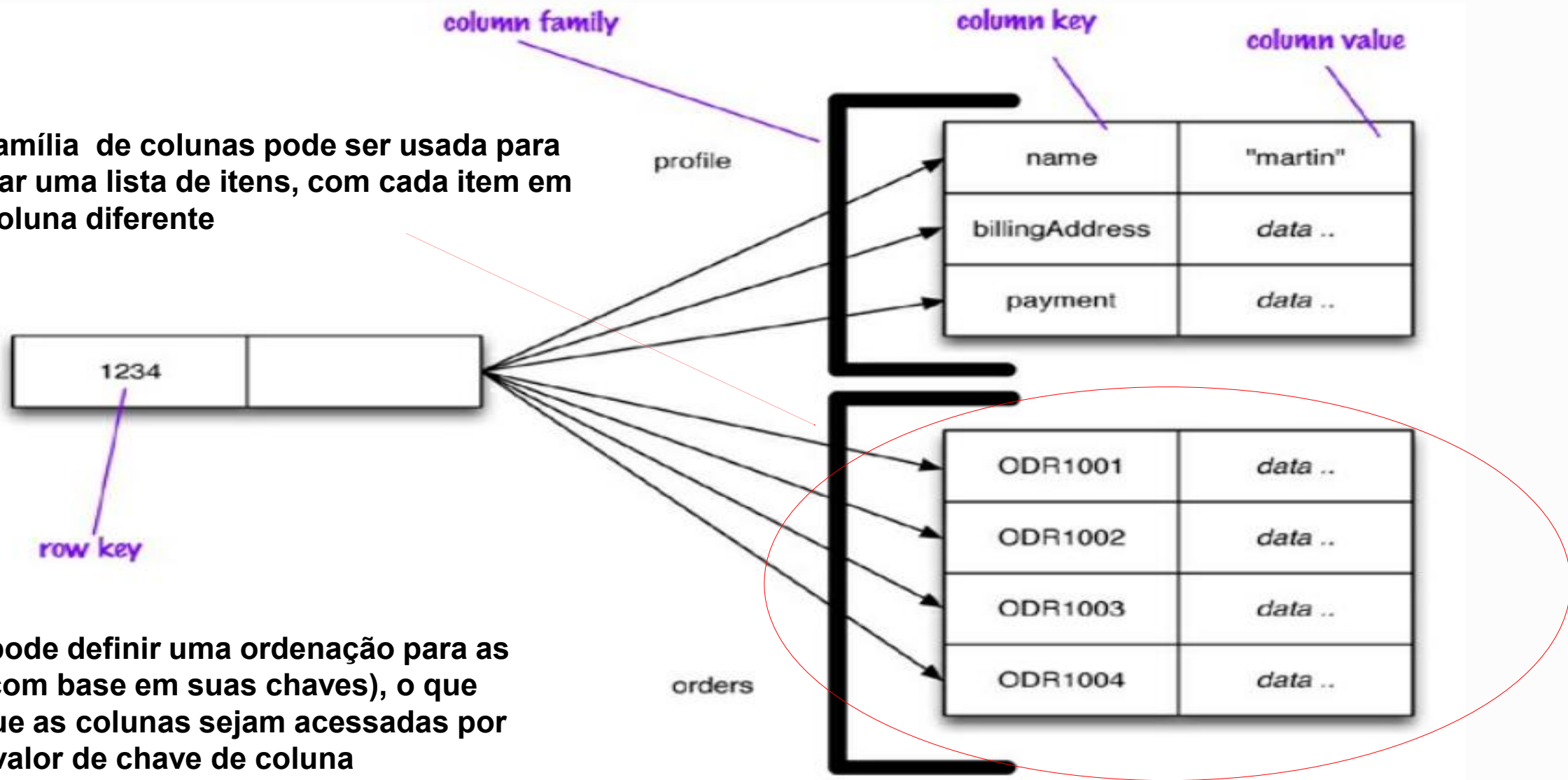
Cada linha armazena os dados de um cliente (do seu perfil e do seus pedidos)



Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Famílias de Colunas – Exemplo

Uma família de colunas pode ser usada para modelar uma lista de itens, com cada item em uma coluna diferente



A família pode definir uma ordenação para as colunas (com base em suas chaves), o que permite que as colunas sejam acessadas por faixas de valor de chave de coluna

Sistemas de Bancos de Dados de Famílias de Colunas

- O SGBD “enxerga” a organização das colunas em famílias e a usa a seu favor no armazenamento e recuperação dos dados
- Modelo de escalabilidade: dividir tanto linhas quanto colunas em diversos nós
 - Linhas são divididas entre os nós de acordo com o valor da chave
 - Colunas são distribuídas entre os nós de acordo com suas famílias (pré-definidas pelo projetista/programador)

Sistemas de Bancos de Dados de Famílias de Colunas

Exemplos:

- Google BigTable → Precursor
- <https://cloud.google.com/bigtable/>
- Cassandra
- <https://cassandra.apache.org/>
- Hbase
- <https://hbase.apache.org/>
- HyperTable
- <http://hypertable.org/>
- MonetDB
- <https://www.monetdb.org/>

Sistemas de BDs de Famílias de Colunas Populares

☐ include secondary database models

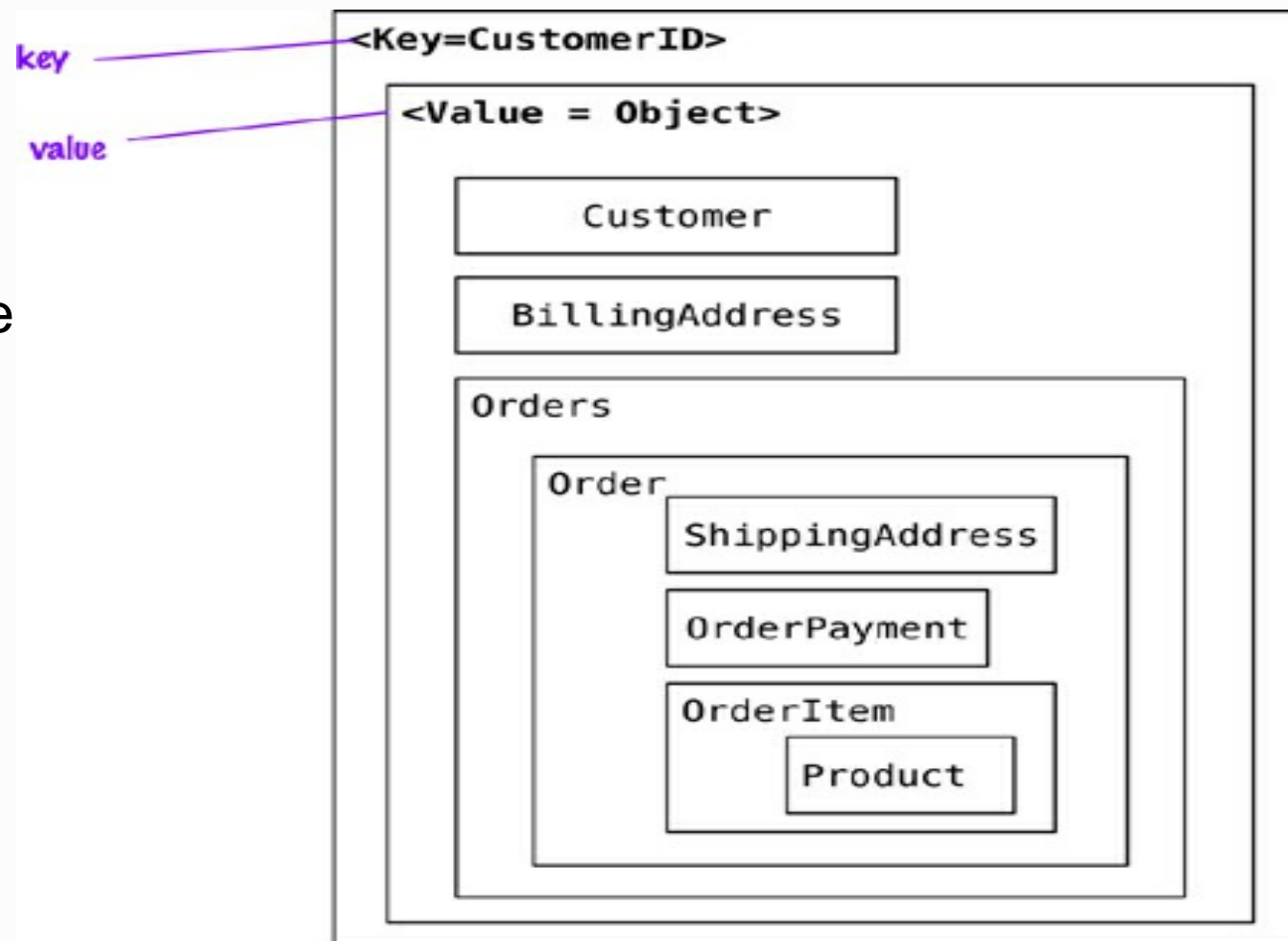
13 systems in ranking, February 2026

Rank			DBMS	Database Model	Score		
Feb 2026	Jan 2026	Feb 2025			Feb 2026	Jan 2026	Feb 2025
1.	1.	1.	Apache Cassandra	Wide column, Multi-model 	101.60	+0.76	-0.98
2.	2.	 3.	Microsoft Azure Cosmos DB	Multi-model 	23.88	-0.92	+1.75
3.	3.	 2.	Apache HBase	Wide column	18.34	-0.61	-6.14
4.	4.	 5.	ScyllaDB 	Wide column, Multi-model 	4.49	+0.20	+0.84
5.	5.	 4.	Datastax Enterprise	Wide column, Multi-model 	3.62	+0.34	-0.60
6.	6.	6.	Microsoft Azure Table Storage	Wide column	2.79	+0.01	+0.07
7.	7.	7.	Google Cloud Bigtable	Multi-model 	2.57	-0.01	-0.13
8.	8.	8.	Apache Accumulo	Wide column	2.53	+0.10	-0.09
9.	9.	9.	Amazon Keyspaces	Wide column	1.10	0.00	+0.13
10.	10.	10.	HPE Ezmeral Data Fabric	Multi-model 	0.80	-0.02	+0.04
11.	11.	11.	Elassandra	Wide column, Multi-model 	0.39	0.00	+0.10
12.	12.	12.	Alibaba Cloud Table Store	Wide column	0.15	0.00	-0.05

<https://db-engines.com/en/ranking/wide+column+store>

Exemplo: BD Chave-valor do Comércio Eletrônico

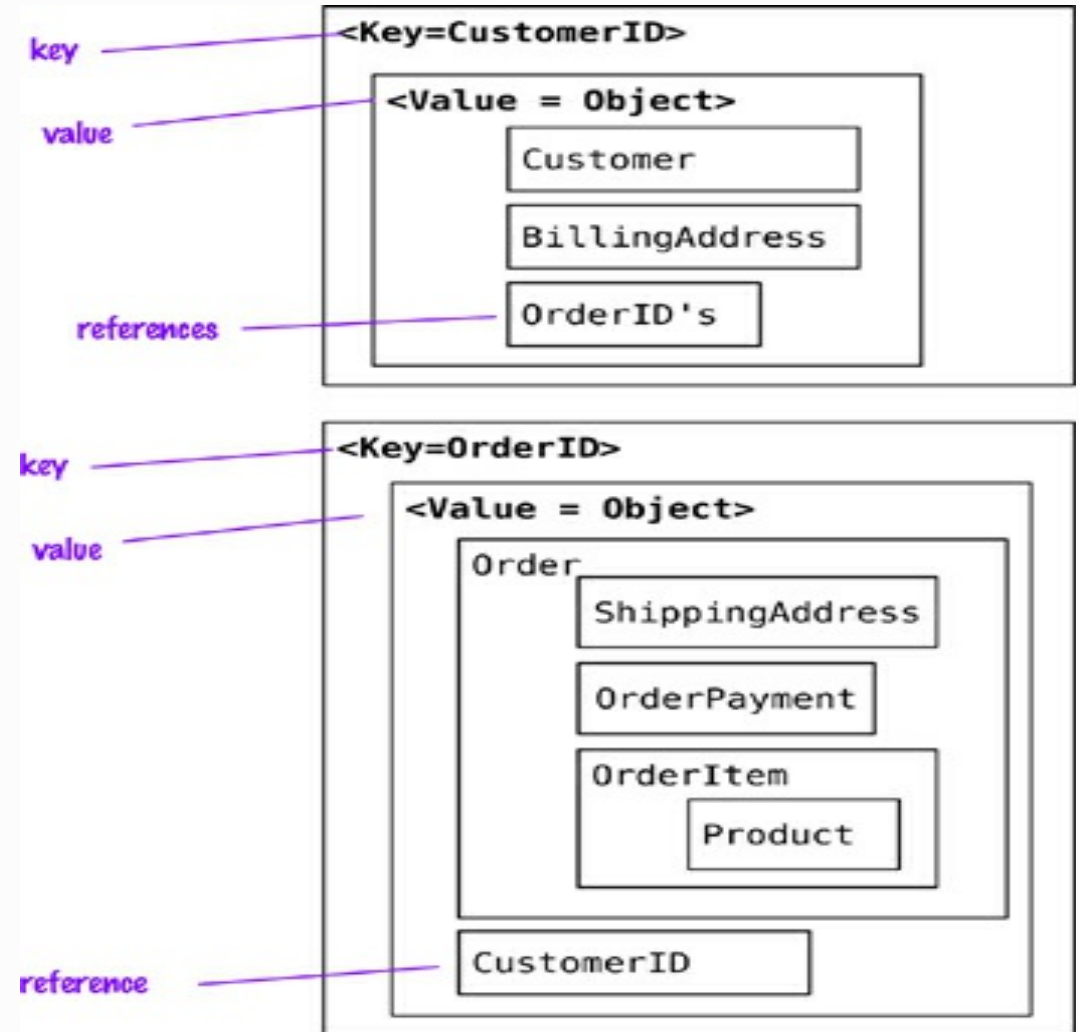
- Leitura de todos os dados de um cliente por meio da sua chave
- Se for necessário consultar dados por pedido, processamento terá que ser feito via aplicação



Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Exemplo: BD Chave-valor do Comércio Eletrônico

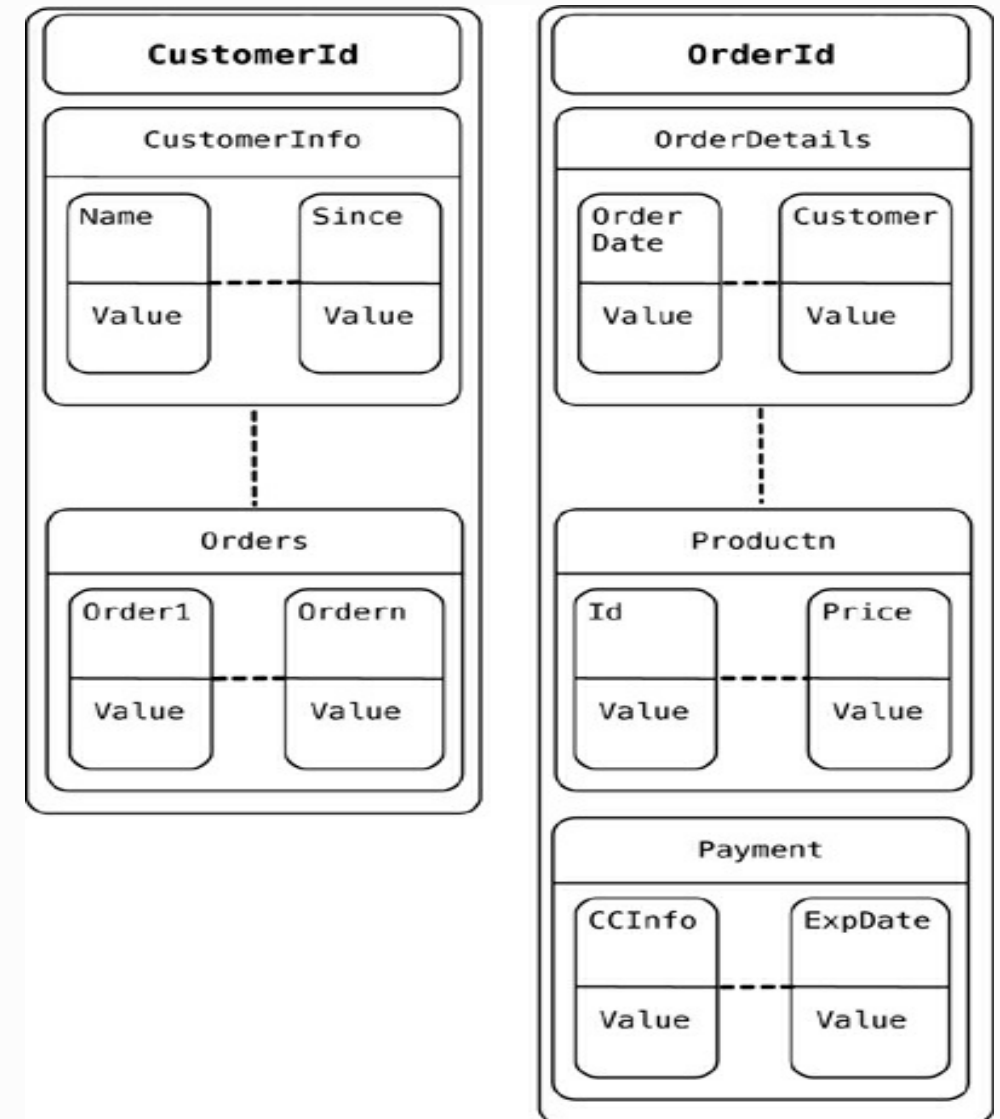
- Alternativa: separar cliente e pedido em dois tipos de agregados diferentes
- Custo adicional: ter que atualizar a lista de pedidos em cliente a cada vez que um pedido for inserido
- Usando um BD de documentos, a lista de pedidos não precisaria ser mantida em Cliente



Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Exemplo: BD de Família de Colunas do Comércio Eletrônico

- Primeiras colunas são carregadas antes
- Algum nível de denormalização é bom em alguns casos



Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Sobre a Ausência de Esquema Fixo

- Liberdade e flexibilidade
 - Estrutura é alterada conforme ganha-se mais conhecimento sobre o projeto (novos atributos são adicionados quando são descobertos)
 - Pode-se deixar de armazenar determinado tipo de dados, sem perder os dados do mesmo tipo que já estão no BD
 - Facilidade de lidar com falta de uniformidade

Sobre a Ausência de Esquema Fixo

- Problema: esquema implícito → embutido nas aplicações que acessam o BD
 - Necessidade de vasculhar códigos para entender o BD
 - SGBD não é “ciente” do esquema e, portanto, não pode fazer otimizações específicas para o BD, nem ajudar a manter a consistência dos dados
 - Manutenção difícil quando múltiplas aplicações acessam o BD

Sobre a Ausência de Esquema Fixo

- Manutenção difícil quando múltiplas aplicações acessam o BD
 - Problema pode ser amenizado com as seguintes estratégias:
 - encapsulando todo acesso ao BD em uma única aplicação (e.g., um serviço web) que será usada pelas demais
 - delineando as partes dos agregados que podem ser acessadas por cada aplicação

Sobre a Ausência de Esquema Fixo

**Com qualquer tipo de modelo de dados,
é preciso cuidado na organização dos dados!**

- Pode-se mudar esquemas de BDs relacionais de forma *ad hoc*:
 - Adicionar novas colunas e tabelas para armazenar dados não uniformes
- É preciso ter controle ao modificar a estrutura de BDs sem esquema fixo:
 - Garantir que tantos os dados no formato novo quanto os dados previamente existentes no formato antigo poderão ser facilmente acessados

Modelos Baseados em Agregados – Resumo

- Modelos Baseados em Agregados
 - BDs chave-valor
 - Agregados opacos, busca por chave
 - BDs de documentos
 - Agregados não-opacos, busca e indexação por conteúdo
 - BDs de famílias de colunas
 - Armazenamento leva em conta a estrutura interna dos agregados

Sistemas NoSQL – Modelos de Dados

- Diferentes tipos de modelos de dados são usados nos sistemas NoSQL
 - Grande heterogeneidade
- Esses modelos costumam ser divididos nas seguintes categorias:
 - **Chave-valor**
 - **Documentos**
 - **Famílias de colunas**



orientados a
agregados

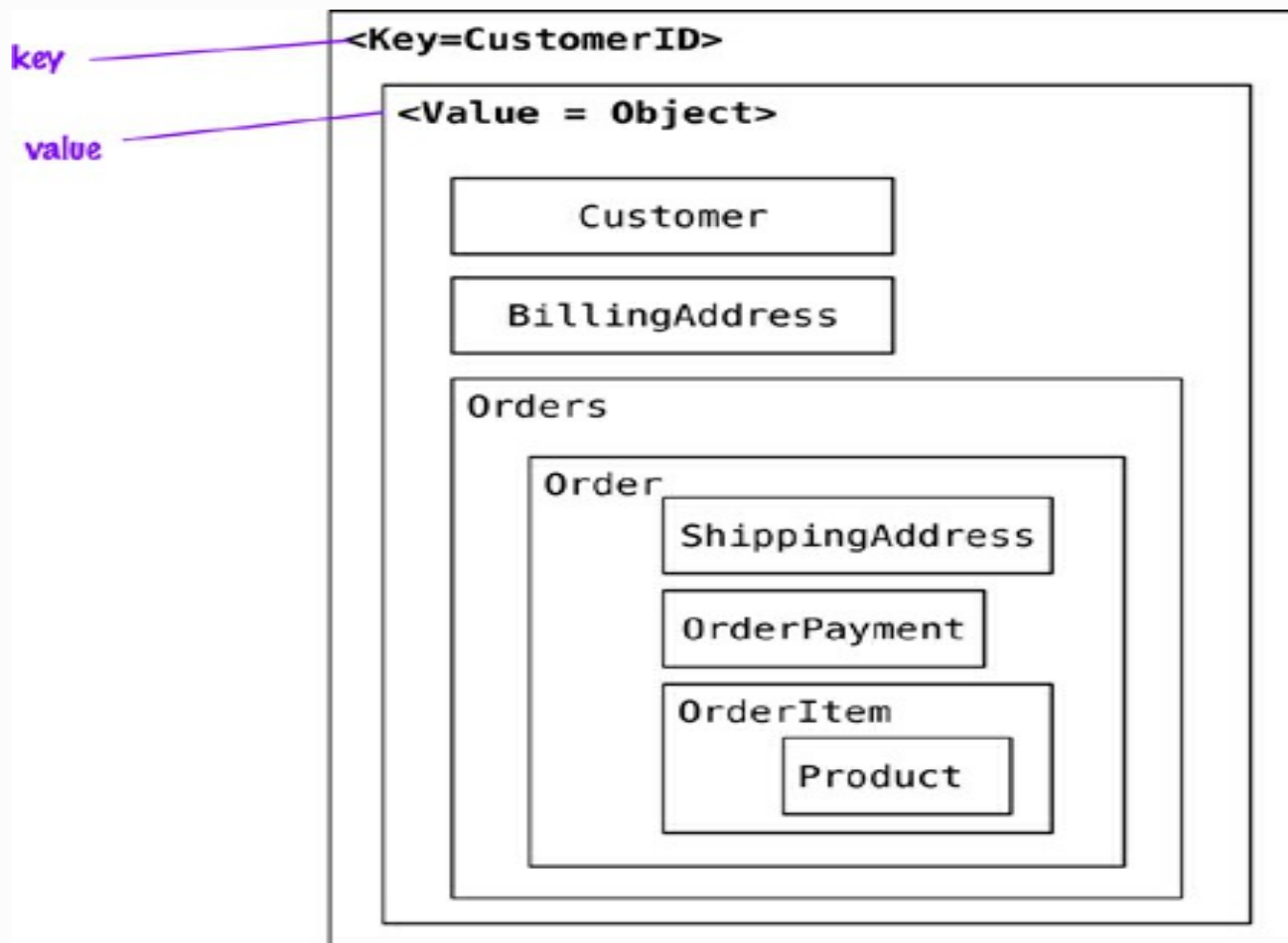
– **Grafos**

Agregados e Relacionamentos

- Agregados são úteis para manter num mesmo lugar dados que são frequentemente acessados juntos
 - Problema quando os mesmos dados são acessados de diferentes formas pelas aplicações
- Ex.: BD do sistema de comércio eletrônico
 - 1) Aplicativo para os clientes
 - → histórico de pedidos por cliente
 - 2) Aplicativo para os lojistas
 - → processamento individual dos pedidos

Agregados e Relacionamentos

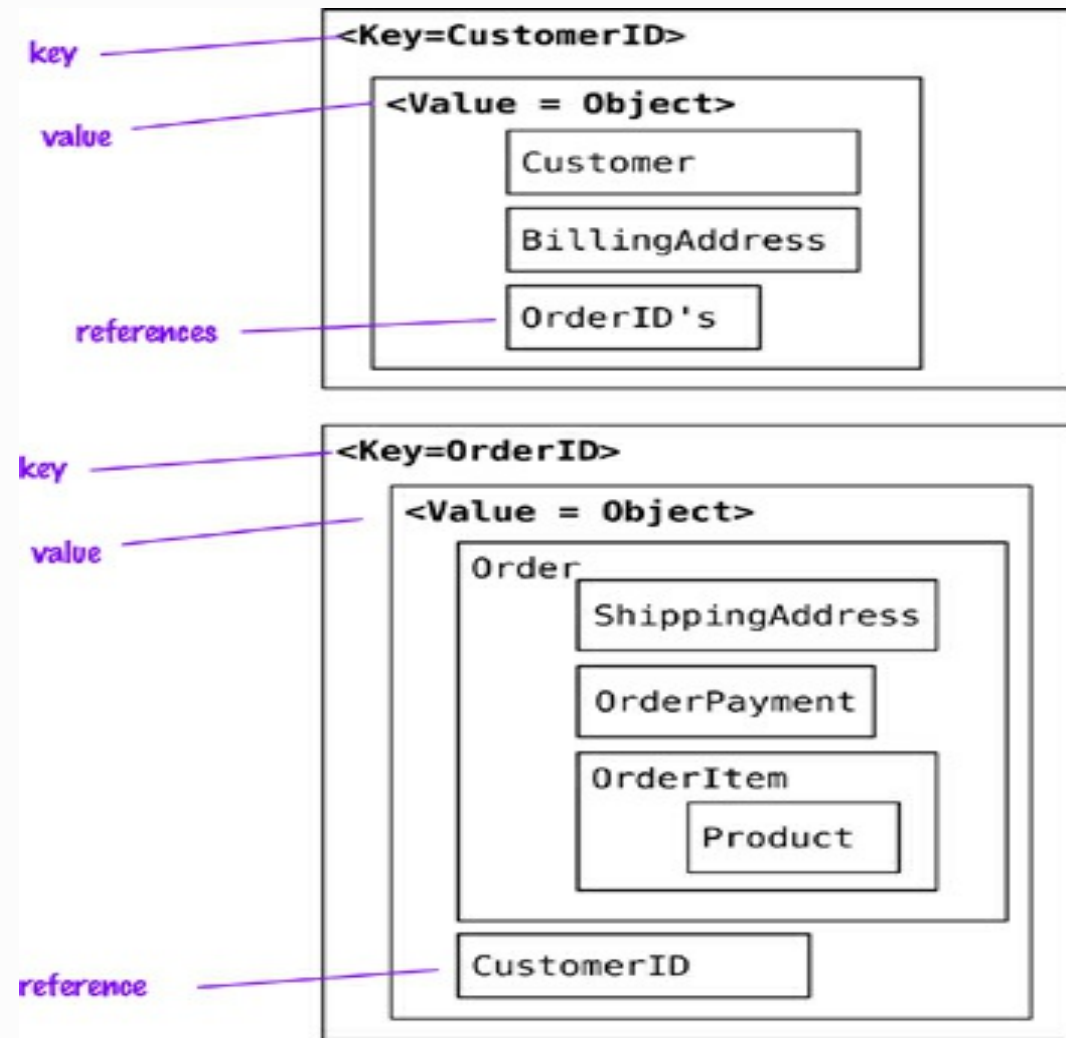
Modelagem de agregado ***Customer*** que favorece a recuperação do **histórico de pedidos por cliente**



Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Agregados e Relacionamentos

Modelagem de agregados ***Customer*** e ***Order*** que favorece o **processamento individual dos pedidos**



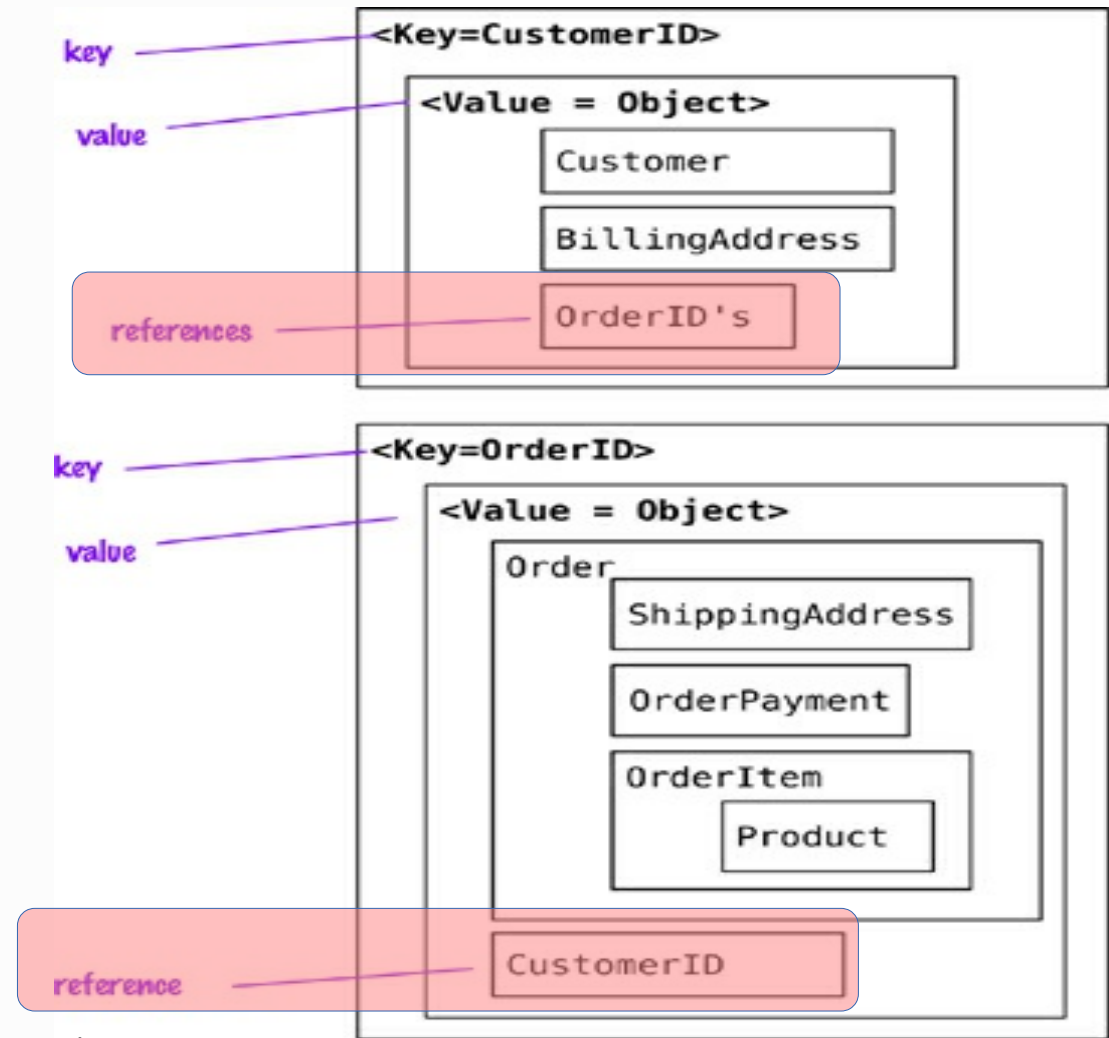
Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Agregados e Relacionamentos

- No caso de agregados separados para pedidos e clientes, é possível criar um **link** (relacionamento) entre pedido e cliente
 - Ex.: incluir o ID do cliente no agregado do pedido
 - Para se obter detalhes do cliente que fez um dado pedido:
 - Recuperação em duas etapas
 - 1) A aplicação submete ao BD uma consulta para recuperar o agregado do pedido
 - 2) Depois de receber a resposta, a aplicação submete uma nova consulta para recuperar o agregado do cliente dono do pedido

Agregados e Relacionamentos

Link (relacionamento)
entre os agregados
Customer e ***Order***



Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Agregados e Relacionamentos

- **Problema 1:** nem todo sistema NoSQL tem funcionalidades para tratar *links* entre agregados
 - Quando o NoSQL é “ciente” de *links*, ele pode criar índices e oferecer funcionalidades para facilitar as consultas
 - Junções e outras operações envolvendo agregados de coleções diferentes podem ser feitas por meio de *MapReduce* (e serem materializadas se conveniente)

MapReduce: modelo de programação para o processamento de grandes volumes de dados de forma paralela e distribuída, em aglomerados de máquinas.

Agregados e Relacionamentos

- **Problema 2:** tratamento das alterações
 - Atomicidade só é garantida em alterações de conteúdo de um único agregado
 - Em alterações envolvendo múltiplos agregados, o tratamento de falhas precisa ser feito pela aplicação

Agregados e Relacionamentos

- Em SGBDs relacionais, essas operações são mais naturais:
 - É possível modificar múltiplos registros em uma única transação com garantias ACID
 - Relacionamentos simples são expressos formalmente, por meio de chaves estrangeiras
 - Cruzamento de registros relacionados (junções) são expressos por meio da SQL
 - Problema: dificuldade em lidar com consultas envolvendo muitas junções

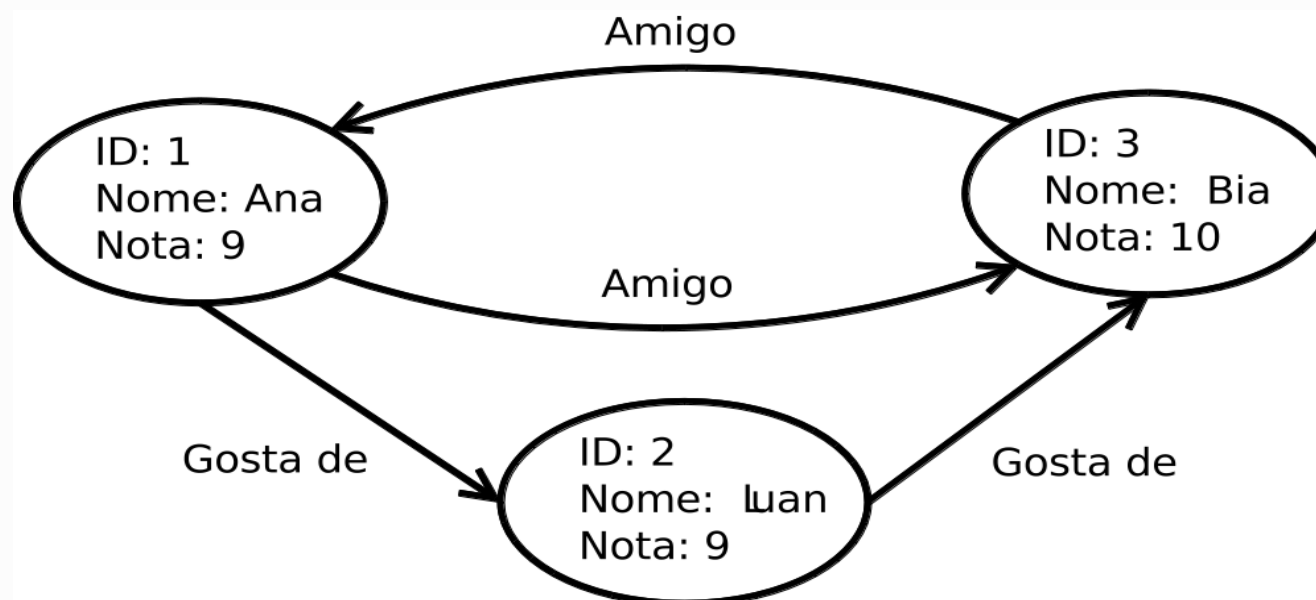
Banco de Dados de Grafos

- Bancos de dados orientados a agregados → grandes registros com conexões simples entre si
 - Motivados pela necessidade de se usar aglomerados de máquinas para o gerenciamento dos dados
- **Bancos de dados de grafos → registros pequenos com interconexões complexas**
 - Motivados pela ineficiência dos SGBDs relacionais para representar relacionamentos complexos e “percorrê-los” em consulta

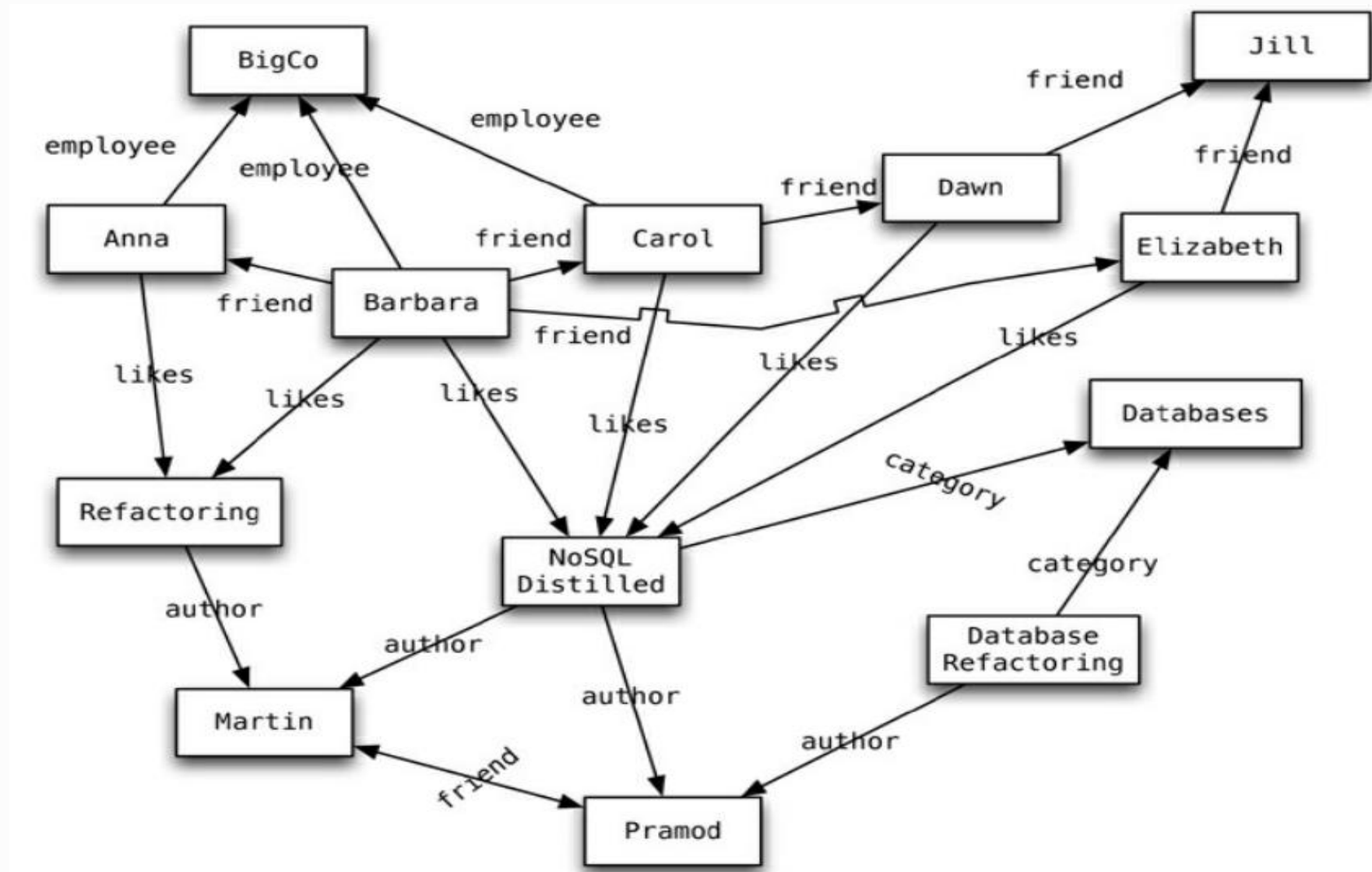
Bancos de Dados de Grafos

- Estrutura de dados de nós interconectados por arcos
- Modelo de dados geral: **nós identificados (entidades)** e **arcos rotulados (relacionamentos)**
- Em alguns sistemas, nós e arcos também podem ter **propriedades** (valores simples ou objetos), geralmente sem esquema fixo

Exemplo de grafo rotulado de propriedades



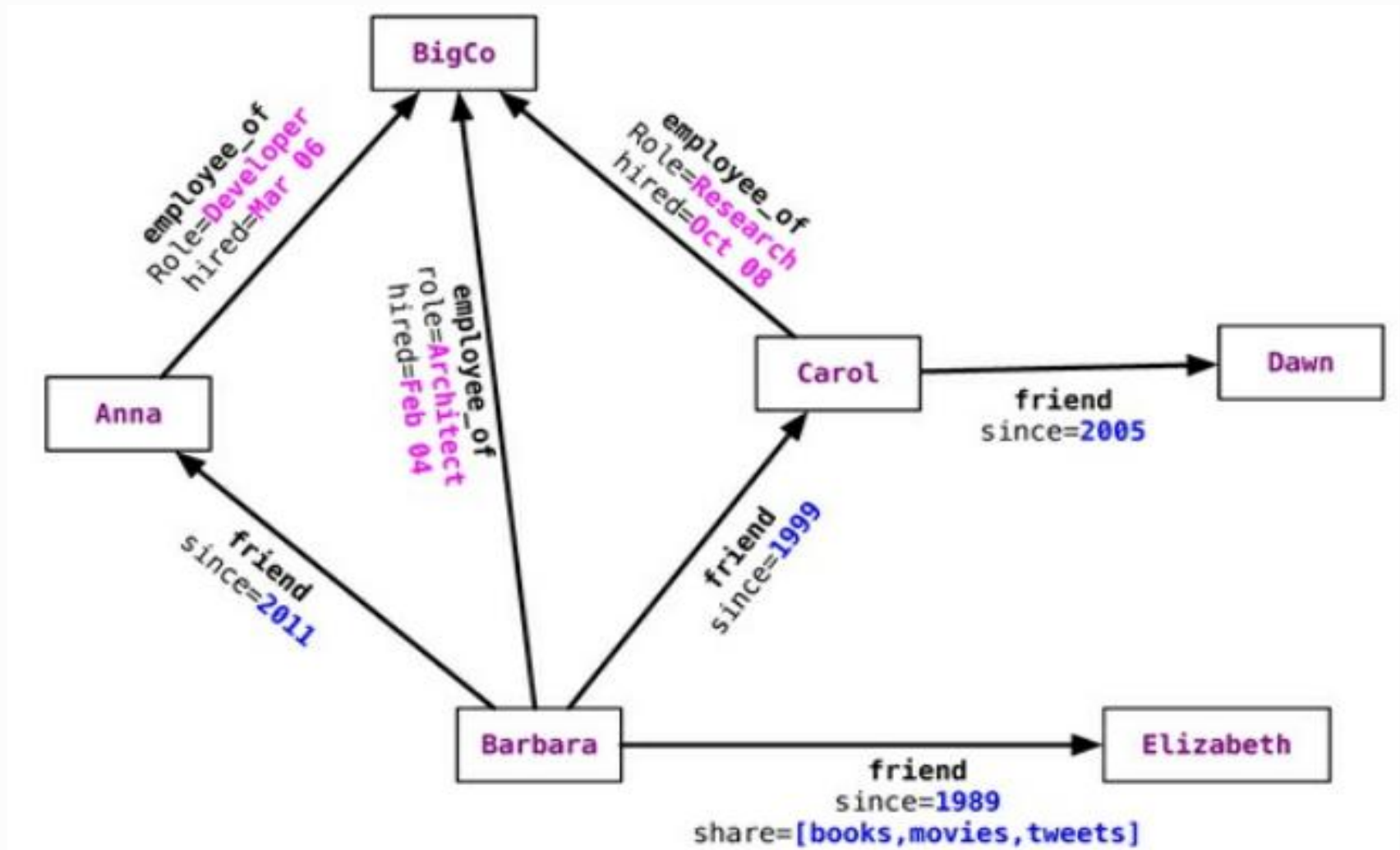
BD de Grafo – Exemplo



Fonte da imagem: NoSQL
Distilled. Sadalage, P. J., &
Fowler, M. (2013)

BD de Grafo – Exemplo 2

Nós com Propriedades



Fonte da imagem: NoSQL
Distilled. Sadalage, P. J., &
Fowler, M. (2013)

Consultas em BDs de Grafos

- Percorrer relacionamentos é uma operação bastante rápida
- Operações de consulta (= percurso) especialmente projetadas para grafos:
 - condições sobre propriedades e rótulos
 - um único passo, expressões de caminho ou recursões completas
 - Exemplos:
 - “Encontre os livros da categoria 'Bancos de Dados' que foram escritos por alguém de quem um amigo meu gosta.”
 - “Encontre todos os amigos (diretos e indiretos) da 'Carol'”

Consultas em BDs de Grafos

- Num BD de grafos, o maior custo computacional associado às operações de consulta fica na operação de inserção no BD
 - Esse custo vem da indexação dos nós
 - Estratégia é vantajosa quando bom desempenho nas consultas é o principal requisito

Consultas em Bancos de Dados de Grafos

- A construção das respostas para as consultas se inicia em um ou mais nós, a partir dos quais os relacionamentos começam a ser percorridos
 - A indexação dos nós é muito importante!
 - Cada nó do grafo guarda referências diretas para seus nós vizinhos, sem precisar consultar um índice global para atravessar o grafo
 - Estratégia chamada de ***index-free adjacency*** (adjacência sem índice)
- Exemplo: "Obtenha todas as coisas de que tanto a Ana quanto a Bárbara gostam"
 - Passo 1: *lookup (busca)* por ID (Ana ou Bárbara)
 - Passo 2: percorrer os relacionamentos diretos dos dois nós iniciais

Bancos de Dados de Grafos

- Sistemas de BDs de grafos costumam implementar transações ACID
- São considerados NoSQL por:
 - Não usarem modelo relacional
 - Terem se popularizado na mesma época que os demais tipos de sistemas NoSQL

Sistemas de BDs de Grafos mais Populares

☐ include secondary database models

43 systems in ranking, February 2026

Rank			DBMS	Database Model	Score		
Feb 2026	Jan 2026	Feb 2025			Feb 2026	Jan 2026	Feb 2025
1.	1.	1.	Neo4j	Graph	49.26	-0.89	+3.92
2.	2.	2.	Microsoft Azure Cosmos DB	Multi-model ⓘ	23.88	-0.92	+1.75
3.	3.	3.	Aerospike	Multi-model ⓘ	3.66	-0.11	-1.43
4.	4.	↑ 5.	ArangoDB	Multi-model ⓘ	3.47	+0.15	+0.56
5.	5.	↓ 4.	Virtuoso	Multi-model ⓘ	2.85	-0.09	-0.29
6.	6.	6.	OrientDB	Multi-model ⓘ	2.56	-0.01	-0.16
7.	7.	7.	GraphDB	Multi-model ⓘ	2.51	+0.04	-0.12
8.	8.	↑ 9.	Amazon Neptune	Multi-model ⓘ	2.46	+0.03	+0.33
9.	9.	↑ 12.	NebulaGraph	Graph	2.21	-0.13	+0.64
10.	10.	↓ 8.	Memgraph	Graph	1.95	-0.08	-0.59
11.	11.	↓ 10.	JanusGraph	Graph	1.89	+0.01	+0.20
12.	12.	↓ 11.	Stardog	Multi-model ⓘ	1.76	0.00	+0.15

<https://db-engines.com/en/ranking/graph+dbms>

Linguagens de Consulta para BDs de Grafos

- Não há linguagens de consulta padronizadas para BDs de grafos
 - Cada sistema oferece a sua linguagem e/ou API para manipulação dos dados
- Algumas propostas já são implementadas em mais de um sistema:
 - Gremlin
 - <http://tinkerpop.apache.org/gremlin.html>
 - Cypher
 - <https://neo4j.com/developer/cypher-query-language/>
 - SPARQL
 - <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>

Exemplos de Consultas Escritas em Cypher

- **Find the title of the movies directed by Tom Hanks**

```
MATCH (:Person {name: 'Tom Hanks'})-[:DIRECTED]->(movie:Movie)
RETURN movie.title
```

- **Find which people are friends of someone who works for Neo4j**

```
MATCH (p:Person)-[r:IS_FRIENDS_WITH]->(friend:Person)
WHERE exists((p)-[:WORKS_FOR]->(:Company {name: 'Neo4j'}))
RETURN p, r, friend;
```



Referências Bibliográficas

- Sadalage, P. J., & Fowler, M. (2013). *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Pearson Education.
- **Leituras extras indicadas:**
- Frozza, A. , Schreiner, A., & Mello, R. *Projeto de Bancos de Dados NoSQL* - material de um minicurso do Simpósio Brasileiro de Bancos de Dados (SBBD 2022)
- <https://sol.sbc.org.br/livros/index.php/sbc/catalog/view/103/461/733-1>
- Atzeni, P., Bugiotti, F., Cabibbo, L., & Torlone, R. (2020). *Data modeling in the NoSQL world*. Computer Standards & Interfaces, 67, 103149.
- <https://dl.acm.org/doi/abs/10.1016/j.csi.2016.10.003>

Modelos de Distribuição de Dados em Sistemas NoSQL

Baseado no livro *NoSQL Distilled – A Brief Guide to the Emerging World of Polyglot Persistence*, de Pramod Sadalage e Martin Fowler

Modelos de Distribuição

- *Sharding* (particionamento)
 - Escala escritas e leituras
- Replicação
 - Escala somente leituras
 - Dois modelos possíveis:
 - Primário-Secundário e Par-a-Par
- *Sharding* + Replicação

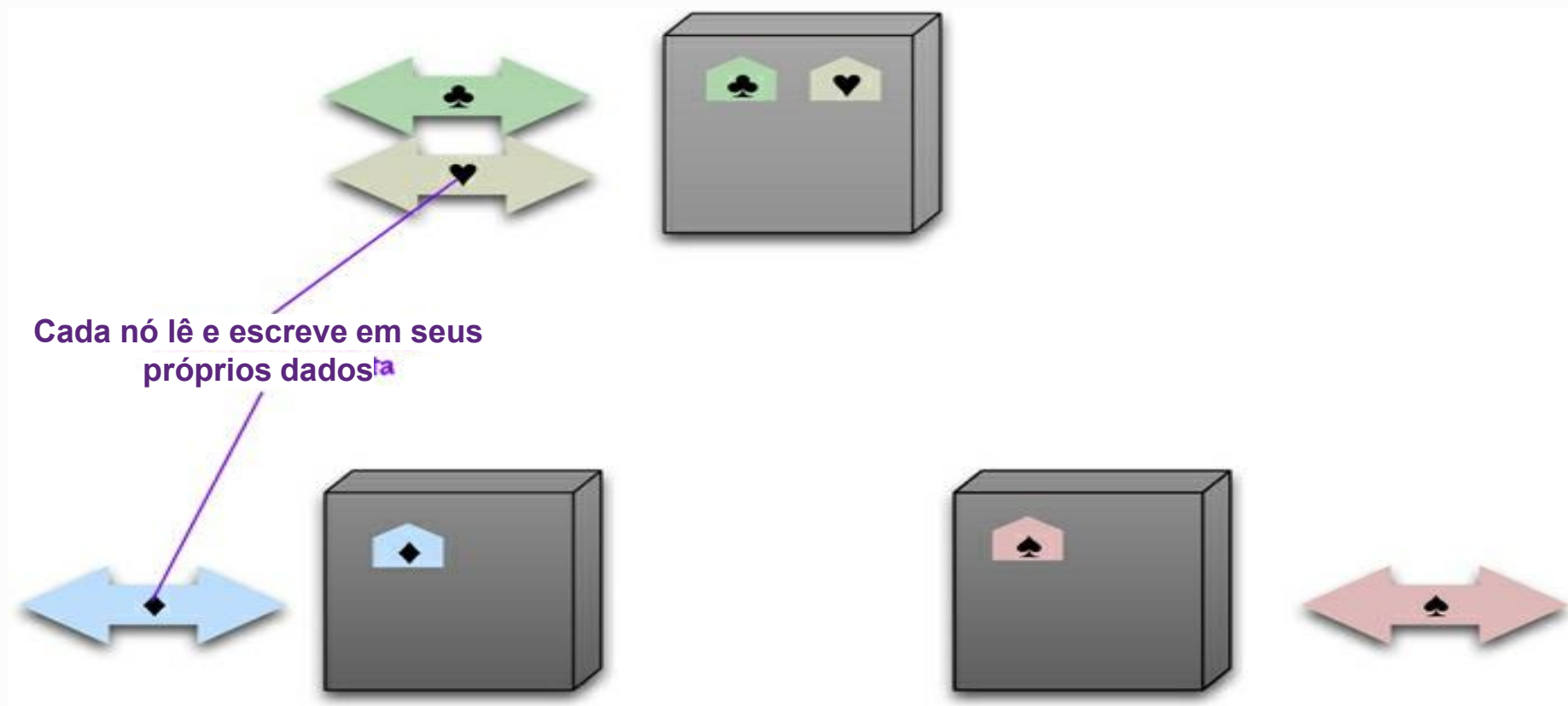
Sharding

- *Sharding* = colocar diferentes partes dos dados em diferentes máquinas
 - Permite paralelismo nos acessos
- Cenário ideal
 - Usuários diferentes se comunicando com nós diferentes
 - Cada usuário precisa acessar os dados de um único nó para realizar uma operação e, por isso, recebe a resposta rapidamente
 - Carga de processamento é balanceada entre os nós (= cada nó recebe a mesma quantidade de trabalho)

Depende de:

- Boa modelagem dos agregados
- Boa estratégia de distribuição dos dados

Sharding - Ilustração



Fonte da imagem: NoSQL
Distilled. Sadalage, P. J., &
Fowler, M. (2013)

diferentes partes dos dados em diferentes nós

Sharding - Estratégias de Distribuição

- **Por localidade**

- Colocar agregados em nós cuja localização física é mais próxima dos seus usuários mais frequentes
- Na instanciação de VMs em nuvens públicas, é possível escolher a região da máquina instanciada (relacionada à localização do *data center*)

- **Por carga**

- Colocar nos nós volumes de dados ou de trabalho (= acessos esperados) parecidos
- A carga pode variar ao longo do tempo

- **Por tipo de acesso**

- Exemplo: colocar num mesmo nó os agregados que podem ser lidos em sequência (geralmente, ordenados segundo algum critério)

Sharding - Implementação

- Historicamente, era parte da lógica da aplicação
 - Definição dos *shards* embutida no código
 - Exemplo: colocar os dados dos clientes com inicial do nome entre A e D em um *shard*, entre E e H em outro *shard*, etc.
 - Redirecionamento das consultas para os *shards* corretos
 - Rebalanceamento requer mudança do código e migração dos dados
- Atualmente, muitos sistemas NoSQL oferecem a funcionalidade de **auto-sharding**
 - Responsabiliza-se por distribuir os dados nos *shards* e direcionar os acessos para os *shards* corretos

Sharding - Resiliência

- Quando usado sem replicação de dados, *sharding* não provê muita resiliência
 - Uma falha em um nó deixa os dados nesse nó inacessíveis (da mesma forma que ocorreria caso em BD estivesse num único servidor)
 - Dados nos demais nós ficam acessíveis
 - Mas nem sempre ter um BD parcialmente acessível é uma opção válida
 - *Clusters* costumam usar máquinas mais simples, mais susceptíveis a falhas
 - Com um único servidor, é mais fácil manter o BD no ar e funcionando

Replicação

- Modelos de replicação de dados:
 - **Primário-Secundário**
 - também conhecido como Mestre-Escravo
 - **Par-a-Par**

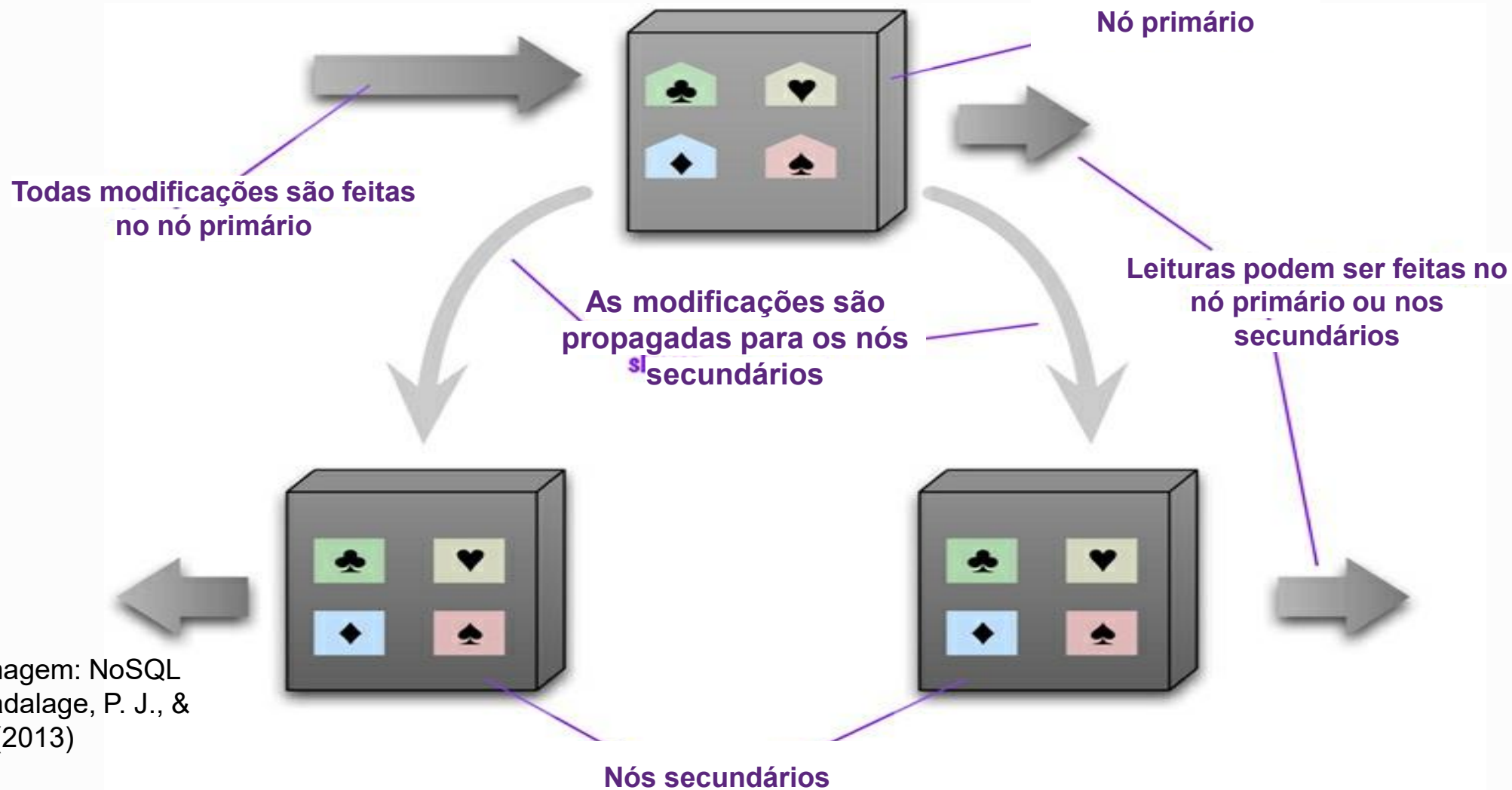
Replicação

Distribuição Primário-Secundário

- Um nó é designado como **primário** (ou mestre)
 - O nó primário é a fonte oficial dos dados, responsável por processar as operações de modificação
- Demais nós são chamados de **secundários** (ou escravos)
 - Um processo de replicação sincroniza os nós secundários com o nó primário

Replicação

Distribuição Primário-Secundário



Fonte da imagem: NoSQL Distilled. Sadalage, P. J., & Fowler, M. (2013)

Replicação

Distribuição Primário-Secundário

- Provê escalabilidade para BDs intensivos em operações de leitura
 - Mais nós secundários → mais capacidade de processamento de leituras
 - O processamento das escritas é limitado pela capacidade do nó primário

Replicação

Distribuição Primário-Secundário

- Também provê **resiliência para as leituras**
 - Em caso de falha no nó primário, leituras podem continuar sendo feitas nos nós secundários
- Quando o nó primário falha, um dos nós secundários é indicado para assumir seu lugar (de forma manual ou automática, por meio de votação entre os nós)
 - Durante o processo, operações de escrita não podem ser feitas
 - O processo da recuperação do nó primário é rápido, já que os secundários são réplicas do primário

Replicação

Distribuição Primário-Secundário

- Primário-secundário pode ser usado também para para manter *backups* dinâmicos para BDs mantidos em um único servidor
 - Nesse caso, todas as escritas e leituras são sempre direcionadas para o nó primário, enquanto os secundários apenas mantêm cópias dos dados

Replicação

Distribuição Primário-Secundário

- Prós:
 - Escalabilidade para leitura
 - Resiliência em caso de falhas nos nós secundários
- Contras:
 - Nó primário como gargalo para a escrita
 - Nó primário como ponto crítico de falha
 - Possibilidade de leituras de dados desatualizados de nós secundários
 - Problema chamado de *inconsistência temporária*

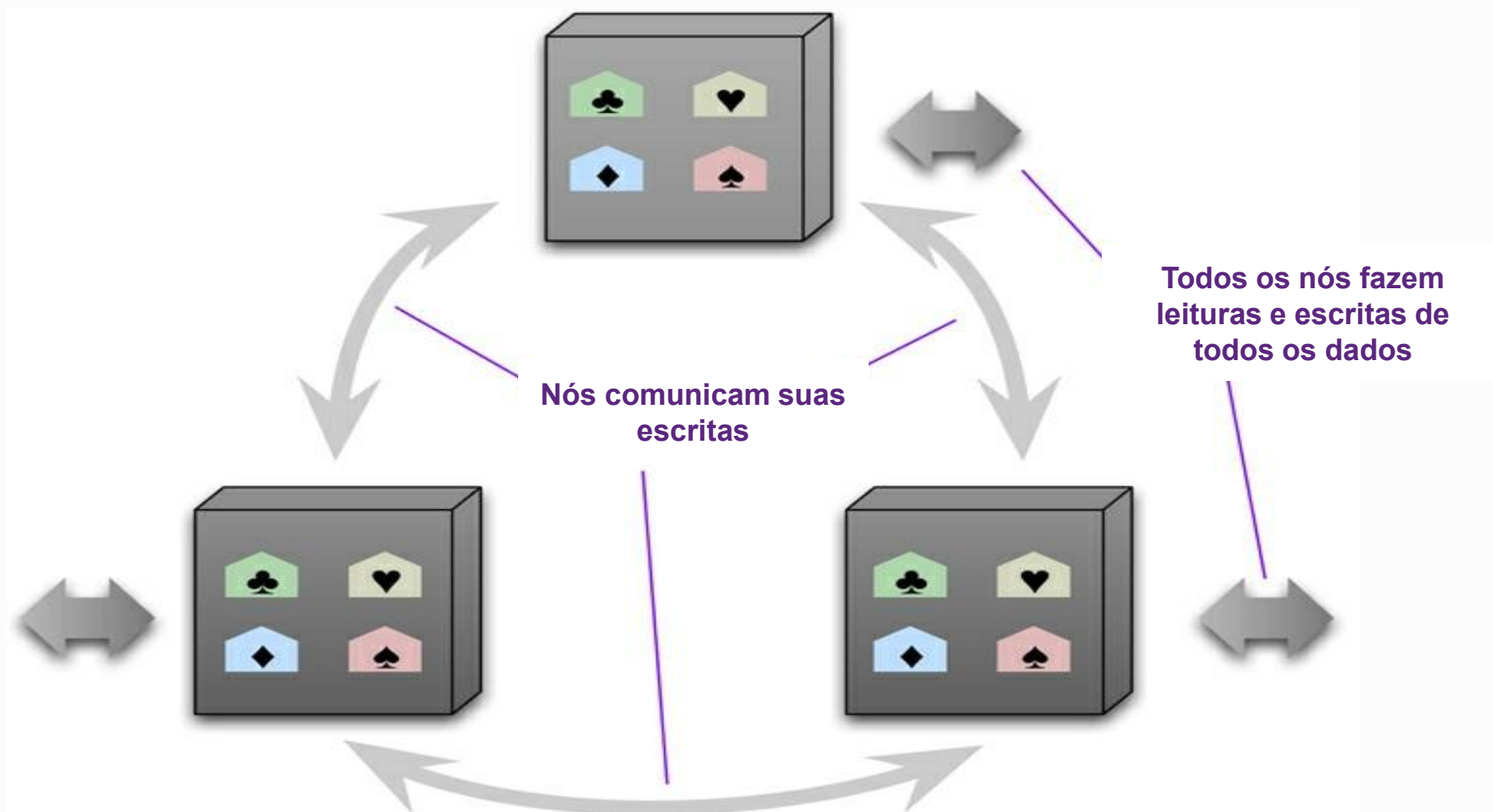
Replicação

Distribuição Par-a-Par

- Todos os nós processam operações de leitura e escrita
 - A falha de um nó não impede a realização de nenhum tipo de operação
- Quanto mais nós, mais capacidade de processamento de leituras e escritas
- Problema: Possibilidade de conflitos do tipo escrita-escrita
 - Duas operações de escrita de um mesmo item de dado ocorrendo ao mesmo tempo em nós diferentes
 - Esse é um tipo de inconsistência permanente

Replicação

Distribuição Par-a-Par



Replicação

Distribuição Par-a-Par

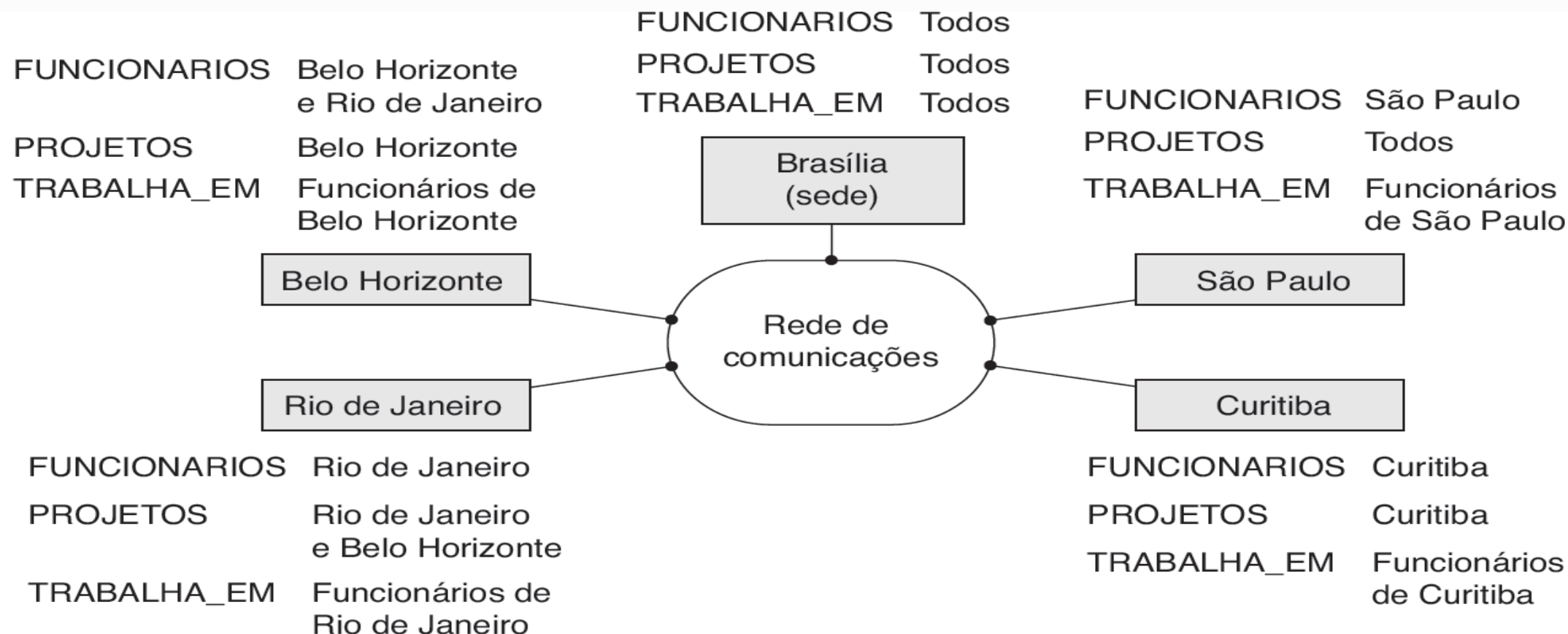
- Estratégias para lidar com conflitos do tipo escrita-escrita:
 - Coordenar as escritas por meio de comunicação entre os nós de réplicas, para evitar conflitos
 - Custo: alto tráfego na rede
 - Benefício: garantia forte de consistência
 - Obs.: Nem todas as réplicas precisam "concordar" com uma escrita, apenas a maioria delas
 - Não evitar conflitos
 - Custo: risco de inconsistência
 - Benefício: melhor desempenho que se pode ter nas escritas

Barganha: Consistência X Disponibilidade

Uso Combinado de *Sharding* e Replicação

- Pode-se ter vários nós primários
- Cada item de dado só tem um nó primário
- Um mesmo nó pode ser primário para alguns itens de dados e secundário para outros
- Exemplo de configuração comum para BDs orientados a famílias de documentos:
 - *Sharding* + replicação par-a-par
 - Fator de replicação 3 (ou seja, cada *shard* tem 3 cópias)

Uso Combinado de *Sharding* e Replicação – Exemplo



Fonte: Introdução aos Sistemas de Bancos de Dados, 6ª edição. R. Elmasri, S. Navathe



Referências Bibliográficas

- Livro: Sadalage, Pramod J., and Martin Fowler. NoSQL distilled: a brief guide to the emerging world of polyglot persistence. Pearson Education, 2013 - Capítulo 1
- Livro: Navathe, Shamkant B., and Ramez A. Elmasri. Fundamentals of Database Systems (7th edition), Pearson Education, 2016 - Capítulo 24
- Artigo: "Stonebraker, Michael. "SQL databases v. NoSQL databases." Communications of the ACM 53.4 (2010): 10-11.
- <http://dl.acm.org/citation.cfm?id=1721659>
- Blog da ACM Sigmod (para quem gosta de assuntos de BD)
- <http://wp.sigmod.org/>

Obrigada!

Profª Kelly Rosa Braghetto | [linkedin.com/in/kelly-rosa-braghetto/](https://www.linkedin.com/in/kelly-rosa-braghetto/)

